

VDR-LLM-Prolog: Prompt Optimization

How to Do More with Less, Faster, Even with Slower Per-Token Processing and More Accurate Results

Registry: [\[HOWL-VDR-15-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → [\[HOWL-VDR-3-2026\]](#) → [\[HOWL-VDR-4-2026\]](#) → [\[HOWL-LLM-1-2026\]](#) → [\[HOWL-VDR-5-2026\]](#) → [\[HOWL-VDR-6-2026\]](#) → [\[HOWL-VDR-7-2026\]](#) → [\[HOWL-VDR-8-2026\]](#) → [\[HOWL-VDR-9-2026\]](#) → [\[HOWL-VDR-10-2026\]](#) → [\[HOWL-VDR-11-2026\]](#) → [\[HOWL-VDR-12-2026\]](#) → [\[HOWL-VDR-13-2026\]](#) → [\[HOWL-VDR-14-2026\]](#) → [\[HOWL-VDR-15-2026\]](#)

DOI: 10.5281/zenodo.20233098

Date: May 2026

Domain: Applied Philosophy / Computational Linguistics

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Abstract

This paper analyzes the token economics of the VDR-LLM-Prolog system specified in the preceding fourteen papers. A conventional language model spends 80 to 95 percent of its generated tokens on infrastructure work — input parsing, state reconstruction, arithmetic computation, logical deduction, data retrieval, formatting, and confidence hedging. Every infrastructure token costs a full forward pass, carries a nonzero error probability, produces no provenance, and consumes context window capacity that then cannot be used for the actual task. The VDR-LLM-Prolog system eliminates infrastructure tokens entirely by offloading each infrastructure function to exact primitives, integer-addressed knowledge base lookups, Prolog evaluation, and grammar templates. The language model generates only judgment tokens (reading content that requires human-level assessment, selecting and sequencing tools) and prose tokens (writing natural language for human consumption). Across seven use cases — SRE incident investigation, legal contract review, medical research synthesis, codebase migration, financial portfolio analysis, customer support knowledge base operation, and academic grading — the system reduces language model token generation by 85 to 97 percent compared to conventional language model processing of the same tasks. The per-operation cost of Q335 exact arithmetic is higher than floating-point arithmetic, but the crossover analysis shows that exact arithmetic would need to be roughly 10,000 times slower per operation before the system breaks even on a single conversational turn, and the margin grows with every subsequent turn because conventional token cost scales with conversation length while primitive cost does not. Several use cases that are impossible for conventional language models — processing

a 1-megabyte JSON payload, summarizing a 10-megabyte document, analyzing 500 financial positions simultaneously, searching 2,000 support articles by indexed query — become routine because data enters through primitives and never passes through the token stream. This paper introduces no new primitives, struct fields, builtins, or modules. It is a pattern-of-use analysis over the existing specification, demonstrating that the architecture specified in VDR-1 through VDR-14 produces a fundamental change in the economics, accuracy, and capability boundaries of language model prompts.

1. The Token Cost Problem

Every interaction with a conventional language model follows the same computational pattern. The user sends a prompt. The model processes the entire conversation history through attention — every prior turn, every fact established, every correction made, every piece of data pasted in. Then the model generates a response token by token. Each token requires a full forward pass through every layer of the model, a full softmax computation over the entire vocabulary, and a sampling decision. The cost of generating one token is the cost of the entire model doing one complete inference step.

The question that has not been systematically analyzed is what those tokens are actually doing. When the tokens are broken down by function, the distribution is revealing.

Consider a moderately complex prompt: an SRE asks a language model to help diagnose a production latency spike, provides some metric data, and asks for analysis. A conventional language model's response might run to 3,000 tokens. Those tokens divide into categories.

Context re-reading consumes no generated tokens directly but dominates the computational cost of the forward pass. Every prior turn in the conversation is re-processed through attention on every generation step. In a 20-turn incident investigation, the attention computation on turn 20 includes all tokens from turns 1 through 19 — potentially tens of thousands of tokens processed through every attention head on every layer, just to reconstruct what is already known.

Input parsing tokens are generated when the model interprets what the user asked. Typo resolution, entity recognition, scope determination — the model handles these implicitly through attention over training data, spending no explicit tokens but consuming attention capacity that could be applied to the actual problem.

State reconstruction tokens appear when the model recaps prior findings. “As we discussed earlier, the latency spike began at 14:23 and correlates with deployment xyz.” These tokens exist solely because the model has no structured state. Every fact must be restated in the token stream to remain accessible. In a long conversation, state reconstruction can consume hundreds of tokens per turn.

Computation tokens appear when the model performs arithmetic, sorting, filtering, or counting. If the SRE pastes metric data showing latency values across 20 endpoints, the model generates tokens to compare values, identify the highest, compute differences, and

assess trends. Every digit it produces is a predicted token. Every comparison is a reasoning chain written out in natural language. A sort that a primitive handles in one call costs dozens of tokens of generated comparison text, with no guarantee of correctness.

Deduction tokens appear when the model reasons through causal chains. “If the latency spike coincides with the deployment, and the error rate also increased, then the deployment is likely the cause.” Each step in the chain is generated text. The model cannot evaluate the deduction — it can only write it out and hope the token prediction produces a valid inference.

Formatting tokens are structural characters in the output: table pipes, JSON braces, markdown headers, list bullets, code indentation. In a JSON response, roughly 55 percent of tokens are structural. In a markdown table, roughly 60 percent. Each structural token costs a full forward pass and has a nonzero probability of being wrong — a mismatched brace, a misaligned column, a dropped comma.

Hedging tokens express uncertainty the model cannot quantify. “Approximately,” “it appears that,” “this could indicate,” “it’s worth noting that,” “based on the available information.” These tokens consume context window capacity and generation cost to express a confidence level that has no computational backing. The model has no mechanism to compute how confident it should be. It generates hedging language because its training data contains hedging language in similar contexts.

Across a typical complex prompt, the distribution is roughly: state reconstruction 15 percent, computation 20 percent, deduction 15 percent, formatting 20 percent, hedging 10 percent, and actual judgment and prose 20 percent. The exact proportions vary by task, but the structural observation holds: the majority of generated tokens serve infrastructure functions, not the user’s actual need. Every infrastructure token has the same computational cost as a judgment token, the same error probability as any other token, and no provenance. The model cannot distinguish its own infrastructure tokens from its judgment tokens. It generates all of them through the same mechanism.

This distribution is not a tuning problem. It is architectural. The model generates infrastructure tokens because it has no other mechanism for performing infrastructure functions. It recaps state because it has no structured memory. It generates digits because it has no arithmetic primitives. It writes comparison prose because it has no sort primitive. It formats structure because it has no grammar system. It hedges because it has no confidence computation. The token stream is the only channel, so everything goes through it.

2. The Prompt Flow

The VDR-LLM-Prolog system processes a prompt through a pipeline where each stage is handled by the appropriate mechanism — primitives for computation, knowledge base operations for state, grammar for structure, Prolog for deduction — and the language model engages only at the stages that require judgment.

The pipeline has seven stages. The first five are infrastructure stages handled entirely by

primitives and knowledge base operations with zero language model tokens. The sixth is the judgment stage where the language model does its actual work. The seventh is the output stage where grammar handles structure and the language model provides prose.

Stage one is input cleanup. The raw user input arrives as a byte sequence. A typo correction knowledge base, persistent at a system-level path and always in scope, contains known corrections and pattern rules. String primitives — B168 `string_contains`, B174 `string_replace`, B163 `string_split` — apply corrections. A classification knowledge base tags each token by type: entity, action, value, reference, noise. The corrected and classified token sequence goes into working data at the session path. Zero language model tokens. The cleanup is deterministic, auditable, and the same corrections apply consistently across all prompts.

Stage two is scope resolution. Each classified token that represents an entity or reference resolves to a knowledge base path through the path registry. B351 `path_resolve` maps a token like “Bob” to its integer ID within the active scope chain. If the active topic is `root.sessions.session_7`, the scope chain searches `session_7`’s knowledge base first, then ancestors to root. Resolution is integer lookup — a hash map from path segments to IDs, then array indexing. If multiple matches exist across scoped knowledge bases, they come back as a typed list with KB addresses and the language model will select in stage six. But the candidates are enumerated and addressed, not hallucinated from pattern matching. Zero language model tokens for the resolution itself.

Stage three is data fetching. Each resolved reference points to data in a knowledge base. B378 `kb_query` pulls facts. B229 `dict_get` pulls from working data dicts. B337 `ring_buffer_read_all` pulls time series. B327 `stack_to_list` pulls investigation history. B335 `lru_get` pulls cached findings. Each is a primitive call — one command token, roughly 8 language model tokens if the language model issues it, but in the automated pipeline the fetch is triggered by the scope resolution results with no language model involvement. The fetched data sits in typed, bounded, addressable containers in working memory. Zero language model tokens.

Stage four is grammar parsing. If the input contains structured content — a table, a code block, a list, a JSON payload, a CSV excerpt — the grammar system recognizes structural tokens and extracts typed fields. A table input becomes a list of dicts with column headers as keys and typed values. A code block becomes a parsed structure with language, content, and metadata. The grammar knows the structure because it has seen it before — grammars are persistent knowledge base fields that inherit through the tree. The language model receives parsed structure, not raw text to interpret through attention. Zero language model tokens.

Stage five is context assembly. The system assembles the information the language model needs to see: the corrected and classified input (compact, already parsed), the resolved references with their KB addresses, the fetched data in typed containers, the grammar-extracted structure, and the current investigation state if any (step counter value, findings in the LRU, investigation path stack contents, budget remaining). This assembled context is structured data at known addresses, not a raw conversation history. The language model reads typed fields, not thousands of tokens of prior turns. Zero language model tokens

for the assembly itself.

Stage six is judgment. The language model reads the assembled context. This is where it engages for the first time. It performs intent recognition — what does the user want? It performs step selection — what primitive, query, or Prolog rule should execute next? It performs formalization — translating the needed step into a command token, a Prolog rule assertion, or a script. These are the tasks that token prediction handles well: pattern matching over structured input, selecting from known vocabularies, writing small targeted programs. The language model spends tokens here — but the input is clean, typed, scoped, and compact. The token cost of judgment scales with the complexity of the actual decision, not with conversation length or data volume.

Stage seven is output. Grammar templates provide all structural tokens for the response format. If the output is a table, grammar provides pipes, headers, alignment, and column formatting. If the output is a report, grammar provides section headers, formatting, and template structure. If the output is code, grammar provides syntax structure. The language model fills content slots with prose — the natural language framing, the explanatory text, the conclusions. Data values render from knowledge base contents through grammar, not through token generation. Confidence renders as an exact fraction computed by propagation rules, not as hedging language. The language model spends tokens only on the prose content that requires human-level language generation.

The result is that language model token generation is concentrated in stages six and seven — judgment and prose. Stages one through five are zero language model tokens. Stage seven's structural component is zero language model tokens. The language model's token budget goes almost entirely to reading pre-structured input and writing prose for human consumption.

3. Token Accounting

A formal accounting model makes the token economics precise. Every token generated by the system falls into one of three categories.

Judgment tokens are tokens the language model generates for tasks that require human-level assessment. Reading a contract clause and determining whether it creates an indemnification obligation. Reading a student essay and assigning a score against a rubric. Assessing an investigation state and deciding what step to take next. Recognizing user intent from structured input. These tokens represent the language model doing what token prediction does well — pattern matching over natural language, applying learned heuristics, making qualitative assessments. Judgment tokens are the irreducible core of language model value.

Command tokens are tokens the language model generates to invoke primitives. A command token is a structured object: a type tag, a primitive name selected from the known vocabulary of roughly 300 names, arguments as dotted-path references or literals, an optional storage path, and an await flag. The average command token costs roughly 8

language model tokens. Command tokens are low-entropy reference selection. The language model is not generating novel syntax or free-form text. It is selecting from known names and pointing at known paths. The error rate on command tokens is structurally lower than on free-form generation because every component is a validated reference.

Infrastructure tokens are tokens a conventional language model generates for parsing, computation, state management, formatting, deduction, and hedging. In the VDR-LLM-Prolog system, infrastructure tokens do not exist. Every function they serve is handled by a mechanism that costs zero language model tokens: parsing by cleanup primitives and grammar, computation by exact arithmetic primitives, state management by knowledge base operations, formatting by grammar templates, deduction by Prolog evaluation, and hedging by exact confidence fraction computation.

The token accounting equation for a conventional language model on a given prompt is: total tokens equals judgment tokens plus infrastructure tokens. The ratio of infrastructure to judgment varies by task but is consistently between 4:1 and 19:1 — infrastructure dominates.

The token accounting equation for VDR-LLM-Prolog on the same prompt is: total language model tokens equals judgment tokens plus command tokens. Infrastructure tokens are zero. Command tokens are a small multiple of the number of primitive calls, at roughly 8 tokens each. The number of primitive calls for a typical prompt ranges from 5 to 50, giving 40 to 400 command tokens.

The savings come from three sources simultaneously. First, infrastructure token elimination: every token that was computation, formatting, state reconstruction, deduction, or hedging is gone. Second, input compression: the language model reads structured typed data instead of raw conversation history, reducing the token count of what it needs to process before generating. Third, output compression: grammar provides structural tokens for free, so the language model's output is predominantly prose content.

The savings are not incremental. They are not a 20 percent reduction from better prompting or a 30 percent reduction from caching. They are an 85 to 97 percent reduction from eliminating entire categories of token generation that the language model should never have been performing.

4. The Crossover Calculation

The objection to exact arithmetic is speed. Q335 integer arithmetic operates on roughly 100-digit numbers. A single integer multiplication on 100-digit numbers is slower than a single floating-point multiplication on a 64-bit value. The question is whether this per-operation slowdown matters when measured against the total system cost.

The cost of a single language model token is substantial. One token requires a forward pass through every attention head in every layer, a full softmax computation over the entire vocabulary (typically 50,000 to 100,000 entries), and all intermediate matrix multiplications in the feedforward blocks. For a model with billions of parameters, one token

involves millions to billions of floating-point operations.

The cost of a single Q335 integer operation is one multiplication of two roughly 100-digit integers (roughly 100 by 100 digit schoolbook multiplication, or faster with Karatsuba), plus one divmod operation of similar magnitude. In total, a few thousand basic arithmetic operations.

The ratio between these costs is enormous. One language model token costs millions of operations. One Q335 arithmetic operation costs thousands. A single language model token buys roughly 1,000 to 10,000 Q335 operations, depending on model size.

Now apply the token accounting. Consider the SRE incident investigation from the use case analysis. A conventional language model spends roughly 3,000 tokens on one turn. The VDR-LLM-Prolog system spends roughly 210 language model tokens on the same turn. The difference is 2,790 tokens.

The primitive calls in that turn involve perhaps a few hundred Q335 arithmetic operations total — some VDR comparisons on metric values, some confidence propagation multiplications, some counter increments (which are plain integer operations, not even Q335). Call it 500 Q335 operations generously.

For the VDR system to break even in total computation cost, those 500 Q335 operations would need to cost as much as the 2,790 saved tokens. Each saved token costs millions of operations. So the 500 Q335 operations would need to each cost millions of basic operations — meaning Q335 would need to be roughly 10,000 times slower per operation than a float multiply before the systems reach parity on a single turn.

Q335 multiplication is not 10,000 times slower than float multiplication. It is perhaps 100 to 1,000 times slower, depending on implementation and hardware. The system has a margin of 10x to 100x on a single turn.

But the margin grows with every subsequent turn, because the two systems scale differently with conversation length.

A conventional language model's cost per turn scales with the conversation history. On turn 1, attention processes the initial prompt. On turn 10, attention processes the initial prompt plus 9 prior turns. On turn 20, attention processes everything from turns 1 through 19. If each turn adds roughly 3,000 tokens of history, turn 20 processes roughly 60,000 tokens through attention before generating even one new token. The generation cost of 3,000 tokens is on top of this attention cost.

The VDR-LLM-Prolog system's cost per turn is roughly flat. State lives in knowledge bases at integer addresses. The language model on turn 20 reads the same amount of structured state as on turn 1 — the current findings in the LRU, the current investigation path on the stack, the current counter values, the current scope. Prior turns are not re-read. They contributed facts to knowledge bases, and those facts are accessible by integer lookup regardless of when they were asserted.

At turn 1, the conventional system costs roughly 3,000 tokens. The VDR system costs roughly 210 language model tokens plus a few hundred primitive operations. Margin: roughly 10x.

At turn 10, the conventional system costs roughly 30,000 tokens of attention processing plus 3,000 generated tokens. The VDR system costs roughly 210 language model tokens plus a few hundred primitive operations plus a few hundred knowledge base lookups (integer array indexing). Margin: roughly 100x.

At turn 20, the conventional system costs roughly 60,000 tokens of attention processing plus 3,000 generated tokens. The VDR system: still roughly 210 language model tokens plus roughly the same primitive and lookup cost. Margin: roughly 200x.

The crossover point — where Q335’s per-operation slowdown would overwhelm the token savings — recedes with every turn. By turn 20, Q335 would need to be roughly 200,000 times slower than float per operation for the systems to break even. No integer arithmetic implementation on modern hardware is 200,000 times slower than float. The Q335 cost is structurally irrelevant at conversation scale.

5. Conversation Length Scaling

The scaling behavior deserves dedicated analysis because it reveals a structural divergence, not merely a quantitative one.

A conventional language model’s relationship with conversation length is adversarial. Every turn makes the next turn more expensive (more history to process through attention), less accurate (more prior tokens competing for attention capacity, more opportunities for the model to lose track of established facts), and more wasteful (more state reconstruction tokens needed as the model recaps to maintain coherence). The model fights against its own conversation history. Long conversations degrade in quality, cost, and reliability simultaneously.

The VDR-LLM-Prolog system’s relationship with conversation length is cooperative. Every turn makes subsequent turns cheaper (facts asserted to knowledge bases are permanently available at constant-time lookup, reducing the need for re-derivation), more accurate (more findings in the LRU, more hypotheses tested and recorded, more evidence accumulated with provenance), and more capable (more Prolog rules available for composition, more grammar patterns established, more data indexed). The system accumulates exact state. It does not re-process history.

The disposable clone mechanism ensures this accumulation never degrades. When a clone approaches drift constraints — maximum turns, context saturation, denominator growth, error rate — it is killed and replaced. Work committed to persistent knowledge bases survives. The fresh clone starts with the frozen snapshot’s clean state plus all accumulated persistent facts. The system’s language model component is always fresh. The system’s knowledge component only grows.

To quantify the divergence, model a 20-turn SRE incident investigation.

Conventional language model: turn 1 costs roughly 3,000 tokens generated plus attention over the initial prompt. Turn 5 costs roughly 3,000 tokens generated plus attention over roughly 15,000 tokens of history. Turn 10 costs roughly 3,000 tokens generated

plus attention over roughly 30,000 tokens of history. Turn 15: roughly 3,000 generated, attention over 45,000. Turn 20: roughly 3,000 generated, attention over 60,000. Total generated tokens across 20 turns: roughly 60,000. Total attention processing across 20 turns: roughly the sum from 0 to 60,000 in increments of 3,000, which is roughly 630,000 token-attention operations. And the quality of turn 20's output is lower than turn 1's because the model's attention is spread across 60,000 tokens of accumulated history, much of which is its own prior infrastructure output — state recaps, formatting, hedging — diluting the signal.

VDR-LLM-Prolog: turn 1 costs roughly 210 language model tokens. Turn 5 costs roughly 210 language model tokens — the investigation state is in the KB, the LLM reads a structured summary of findings, path, and budget. Turn 10: roughly 210 tokens. Turn 20: roughly 210 tokens. The structured state might be slightly larger by turn 20 — more findings in the LRU, more entries in the bitset — but this is a few hundred bytes of structured data, not thousands of tokens of conversation history. Total language model tokens across 20 turns: roughly 4,200. Total attention processing: each turn's attention operates over roughly the same amount of structured input, so roughly 20 times a small fixed amount. And the quality of turn 20's output is higher than turn 1's because the knowledge base contains 19 turns of accumulated, provenance-tagged, exact findings that the LLM can query precisely.

The divergence is superlinear. Conventional cost grows quadratically with turn count (each turn processes all prior history). VDR-LLM-Prolog cost grows linearly (each turn costs the same fixed amount of language model work). The quality curves move in opposite directions: conventional quality decreases with length, VDR-LLM-Prolog quality increases.

6. The Capability Boundary

Token reduction and cost savings apply when both systems can perform the task. There is a more consequential category: tasks that conventional language models cannot perform at all, which the VDR-LLM-Prolog system handles routinely.

The boundary is data volume. A conventional language model can only process data that fits in its context window. The context window is a fixed number of tokens — typically 100,000 to 200,000 for current models, with some extending to 1,000,000. Everything the model needs to work with must be serialized into that token budget: the conversation history, the user's instructions, and all data.

One megabyte of JSON metric data from a prometheus time series export is roughly 250,000 to 500,000 tokens depending on the structure. This exceeds or saturates most context windows. The SRE cannot paste the data in. They must manually truncate, sample, or summarize it before the model can see it. Information is lost before processing begins. The model works on a partial, human-curated subset of the data, introducing selection bias and missing potentially relevant signals.

In the VDR-LLM-Prolog system, the 1MB JSON enters through a network fetch primitive (B423) or file read primitive (B391). The raw bytes never enter the token stream. A parse primitive (B246) produces a typed dict. Filter primitives (B198), sort primitives (B202), and aggregation primitives (B047, B049) operate on the full dataset. The language model sees computed results — the top 20 endpoints by p99, the correlation between deployment events and latency changes, the trend slope per service. These results are compact structured data at knowledge base addresses. The language model's context window is occupied by a few hundred tokens of structured findings, not hundreds of thousands of tokens of raw JSON.

The pattern extends to every data-intensive use case.

A 10-megabyte document — a technical specification, a legal filing, a research report — is millions of tokens. A conventional language model cannot load it. The user must manually segment it, paste sections one at a time, and hope the model maintains coherence across segments while its context window fills with prior summaries. Cross-references between distant sections are lost because the referencing section and the referenced section are never in the context window simultaneously.

In VDR-LLM-Prolog, the document enters through a file read primitive. A parsing script (executed in a sandboxed environment through B410) extracts the document structure — sections, headings, paragraphs, tables, footnotes, cross-references, metadata — as a typed dict tree. The language model reads section names and sizes (maybe 200 tokens of structured input) and plans a summarization strategy. For each section it needs to examine, a dict access primitive (B229) pulls that section's content into working data. The language model reads and summarizes one section at a time, storing each summary in a results dict. Cross-references are preserved in the parsed structure — if section 7 references section 3, that relationship is a typed connection in the knowledge base, queryable at any time. The language model can pull both sections simultaneously if needed. The complete document is accessible by integer address at any point in the conversation without re-parsing or re-reading.

A portfolio of 500 financial positions requires 500 multiplications (quantity times price), 500 subtractions (current minus cost basis), aggregation by sector (group-by, sum within groups, divide by total), and correlation analysis across all position pairs. A conventional language model cannot hold 500 positions in context. Even if it could, performing 500 multiplications through digit-by-digit token prediction would introduce cumulative errors. The VDR-LLM-Prolog system ingests the portfolio through a CSV parse primitive, performs every multiplication as exact VDR arithmetic (one integer multiply plus one divmod per position, zero accumulated error), computes sector allocations as exact fractions, and stores all results at knowledge base addresses. The language model writes the narrative commentary on exact computed results.

A knowledge base of 2,000 customer support articles cannot be searched by a conventional language model. It can be searched by knowledge base query — each article is a node in the tree with indexed facts for product version, error codes, and keywords. A query for articles matching error code 4017 on version 3.2 is two integer comparisons across indexed facts, returning matching article IDs. The language model reads the spe-

cific matched articles, not 2,000 articles.

A codebase of 200 files cannot be analyzed simultaneously by a conventional language model. It can be analyzed by filesystem primitives — read all files, run an analysis script in a sandboxed environment, parse the results into a dependency graph, sort topologically, and process files in dependency order. Each file’s migration state is tracked in a knowledge base. The language model sees one file at a time with its specific migration points, not the entire codebase.

In each case, the capability boundary is not a performance limitation that could be overcome by a larger context window. A 10-million-token context window would make the 1MB JSON case theoretically possible but practically worse — the model would process hundreds of thousands of structural tokens through attention, spending enormous computation to parse brackets and commas that a parse primitive handles in one call. The architectural problem is not window size. It is that data enters through the token stream at all.

7. Use Case Demonstrations

Seven use cases demonstrate the token economics with specific builtin calls, data flows through knowledge bases, and token count comparisons. Each follows the same pipeline: data enters through primitives, structure is extracted by parsers, storage is in typed knowledge bases, computation uses exact primitives, state persists across turns, output structure comes from grammar, and the language model spends tokens only on judgment and prose.

7.1 SRE Incident Investigation

The SRE reports: production is down, latency spiked 10 minutes ago, prometheus is at prom.internal:9090.

A conventional language model cannot connect to prometheus. It says “based on what you’ve described, here are some things to check” and generates 500 tokens of generic troubleshooting advice. The SRE pastes metrics manually. The model re-reads the entire conversation plus pasted data through attention. Over 5 turns of pasting data, asking questions, and receiving analysis, the model generates roughly 15,000 tokens total, much of which is re-reading prior pastes, recapping findings, hedging conclusions, and formatting tables by token prediction. The model’s accuracy degrades as the context window fills with raw metric data diluting its attention.

The VDR-LLM-Prolog system instantiates an SRE incident template knowledge base — the template from VDR-9 Appendix S. This creates a step counter, query counter, hypothesis counter, stall detector counter, findings LRU, sources LRU, attempted steps LRU, investigation path stack, dependencies checked bitset, investigating lock, metrics ring buffer, and remediation queue. All bounded, typed, at integer addresses. Zero language model tokens.

The prometheus URL receives a network grant. B424 network_fetch pulls latency time series for the last hour from the prometheus API. B246 parse_json extracts the data. B254 vdr_from_decimal_string converts every metric value to an exact VDR fraction at the conversion boundary, recording source type, original value, converted value, and maximum error. The time series goes into the metrics ring buffer. Source confidence is tagged at 95/100 per the knowability spectrum. Zero language model tokens for the entire fetch-parse-convert-store pipeline.

The language model's first engagement is assessment. It reads structured state: typed time series summary in the ring buffer, incident template with empty findings, active investigation. It recognizes the pattern and formalizes the first step: pull deployment history from prometheus. Command token: roughly 8 language model tokens. B424 network_fetch executes. Results parse into facts with exact integer timestamps. A deployment occurred 12 minutes ago. The finding goes into the findings LRU with confidence 95/100. The dependencies checked bitset flips the deployment bit. The step counter increments.

The language model assesses again, formalizes: pull resource metrics for the deployed service — CPU, memory, error rate. Three command tokens: roughly 24 language model tokens. Results come back, parse, convert, store. Error rate jumped from [1, 1000, 0] to [47, 1000, 0] at the deployment timestamp. B017 vdr_compare: one primitive call, exact. The finding goes into the LRU. Confidence computed: B003 vdr_mul of 95/100 by 95/100 gives 9025/10000, simplified to approximately 90/100 by the propagation rules for two independent prometheus sources.

The language model formalizes a Prolog hypothesis: root_cause(deployment_xyz) if temporal_correlation(deployment_xyz, latency_spike) and error_rate_increase(deployment_xyz, above_threshold). B376 kb_assert. Prolog evaluates. Both body facts exist. The rule succeeds. Confidence propagates as minimum of premises: 90/100.

The language model checks alternatives. B337 ring_buffer_read_all on infrastructure metrics. No anomalies. Negative finding stored. Dependencies bitset flips the infrastructure bit.

B378 kb_query: root_cause(X) where confidence above 80/100. One result: deployment_xyz at 90/100. Goal satisfied.

Grammar template renders the incident report. The language model writes prose framing: roughly 100 tokens. Time series data renders through grammar from the ring buffer. Confidence is the exact fraction 90/100.

Token accounting — conventional: roughly 3,000 per turn, 15,000 across 5 turns, degrading quality. VDR-LLM-Prolog: roughly 210 total language model tokens across the entire investigation. Breakdown: 80 tokens command tokens, 50 tokens assessment reasoning, 20 tokens Prolog rule formalization, 60 tokens prose output. Reduction: 98.6 percent.

7.2 Legal Contract Review

A lawyer uploads a 50-page contract as contract.docx.

Conventional: the lawyer pastes sections one at a time. The model processes each section through attention, losing cross-references between clauses that appeared in different pastes. Defined terms from clause 1.1 are forgotten by the time the model reads clause 14.2(b). Dollar amounts are generated through digit prediction. The model produces a hedged analysis the lawyer must verify manually. Across 10 turns of pasting and querying: roughly 30,000 tokens, degrading accuracy with each turn.

VDR-LLM-Prolog: B391 `file_read` loads the raw bytes. B410 `execute_python` runs a 5-line python-docx extraction script in a Docker sandbox — the language model writes the script once, roughly 30 tokens. The script outputs JSON. B246 `parse_json` produces a typed dict. B236 `dict_keys` gives the section structure. The language model reads section names: roughly 50 tokens of structured input.

B384 `kb_switch_topic` to `root.sessions.current.contract_review`. The language model formalizes a Prolog rule for defined term extraction: `defined_term(Term, Definition)` if `section(definitions, Text)` and `pattern_match(Text, Term, Definition)`. B376 `kb_assert`. Roughly 20 tokens.

B229 `dict_get` on the definitions section. B163 `string_split` on sentence boundaries. B198 `list_filter` with a predicate matching “means” or “shall mean.” Each match gets B376 `kb_assert` as a `defined_term` fact with clause number as provenance.

The lawyer asks for indemnification obligations and caps. B198 `list_filter` on the clause dict for sections containing “indemnif.” Returns matching clause keys. B229 `dict_get` on each. The language model reads matched clause text — this is judgment work, classifying obligation types and identifying cap amounts. Roughly 200 tokens per clause for 4 matching clauses: 800 tokens.

For each cap amount: B254 `vdr_from_decimal_string` converts the dollar figure to exact VDR. B047 `vdr_sum` across all caps gives total exposure. B004 `vdr_div` computes each obligation as a fraction of total. All exact. B363 `connection_add` links each finding to its source clause.

Grammar template renders the indemnification table. The language model writes a 3-paragraph risk summary: roughly 150 tokens.

Token accounting — conventional: roughly 30,000 tokens across 10 turns. VDR-LLM-Prolog: roughly 30 for the script, 50 reading structure, 20 for the Prolog rule, 800 reading and classifying clauses, 80 in command tokens, 150 in prose. Total: roughly 1,130. Reduction: 96.2 percent.

7.3 Medical Research Synthesis

A researcher needs synthesis across 40 papers on a drug interaction, providing DOIs.

Conventional: the model can hold perhaps 3 papers in context simultaneously. It processes them sequentially, losing details of paper 7 by the time it reads paper 31. Contradictions between distant papers go undetected. Effect sizes are generated from memory of what the model read earlier. Statistical aggregation is performed through digit pre-

diction. The researcher receives a synthesis they cannot trust. Across multiple sessions totaling perhaps 40 turns: roughly 80,000 tokens.

VDR-LLM-Prolog: B248 `parse_csv` on the DOI list. For each DOI, B424 `network_fetch` retrieves metadata from the crossref API in a Docker environment with network grant. B246 `parse_json`. Each paper gets a KB node under `root.sessions.current.synthesis.papers.paper_N`.

For each paper's structured abstract, the language model reads the results section and extracts findings — this is judgment work, roughly 100 tokens per paper. For each finding, B254 `vdr_from_decimal_string` converts study size, effect size, confidence interval bounds, and p-value to exact VDR fractions. B376 `kb_assert` stores each as a provenance-tagged fact: `fact(effect_size, paper_12, [23, 100, 0])`.

After all 40 papers: B378 `kb_query` collects all `effect_size` facts. B052 `vdr_weighted_sum` with study sizes as weights gives exact weighted mean effect. B102 `vdr_stat_variance` gives exact heterogeneity. B105 `vdr_stat_percentile` gives distribution characteristics.

Contradiction detection: the language model formalizes a Prolog rule — `contradicts(A, B)` if effect sizes have opposite signs and both p-values are below 5/100. B376 `kb_assert`. Prolog evaluates across all paper pairs. Contradictory pairs stored as connections. Roughly 40 tokens for rule formalization.

Confidence propagation: B003 `vdr_mul` and B002 `vdr_sub` compute the multiple-sources-agree formula for consistent findings. Conflict penalty formula applies to contradictory pairs. All exact fractions.

The language model reads structured aggregates and writes the synthesis narrative. Grammar template provides report structure. Roughly 500 tokens of prose.

Token accounting — conventional: roughly 80,000 tokens. VDR-LLM-Prolog: roughly 4,000 reading paper results sections, 200 command tokens, 40 Prolog rules, 500 prose. Total: roughly 4,740. Reduction: 94.1 percent. Every number traces to a specific paper at a specific KB address.

7.4 Codebase Migration

A team needs to migrate 200 Python files from Python 2 to Python 3.

Conventional: the developer pastes one file at a time. The model suggests changes that may break downstream files it has never seen. It cannot run tests. It cannot lint. After 50 turns, 15 files are done with 3 regressions introduced. Roughly 100,000 tokens for partial completion.

VDR-LLM-Prolog: Docker environment with filesystem grant on the project directory and execute grant for python and pytest.

B405 `filesystem_tree` on the project root. B198 `list_filter` for .py files: 200 files. For each, B391 `file_read`.

The language model writes a migration analysis script once — roughly 50 tokens. B410 `execute_python` runs it on all 200 files using the `ast` module to identify Python 2 patterns: print statements, except syntax, unicode literals, division behavior, removed modules. Output is JSON. B246 `parse_json`. B208 `list_group_by` on migration category: 47 files with print statements, 12 with unicode issues, 8 with exception syntax, 3 with removed dependencies, 23 with integer division.

The language model writes an import analysis script — roughly 50 tokens. B410 `execute_python` builds the dependency graph. B285 `graph_from_edges`. B295 `graph_topological_sort` gives migration order. Stored in a queue.

For each file in order: B311 `queue_pop`. B229 `dict_get` for its migration points. B189 `list_slice` pulls the flagged lines with context. The language model reads specific lines and writes replacement code — judgment work, roughly 30 tokens per migration point. B392 `file_write` writes the modified file. B413 `execute_pytest` runs the test suite. Results parsed. B298 `counter_inc` on `files_migrated` or `tests_failed`.

If tests fail, the failure goes into the `attempted_steps` LRU. The language model backtracks. Prior file states are in the KB, not in conversation history.

After all files: B410 `execute_python` runs the full test suite. B422 `filesystem_checksum` on every modified file for audit.

Token accounting — conventional: roughly 100,000 tokens for partial completion with regressions. VDR-LLM-Prolog: roughly 100 for scripts, 6,000 across 200 files writing replacement code, 400 command tokens, 200 backtracking reasoning. Total: roughly 6,700 for complete migration with test verification. Reduction: 93.3 percent. Complete versus partial.

7.5 Financial Portfolio Analysis

An analyst has 500 positions and needs risk analysis.

Conventional: the analyst cannot load 500 positions. They paste a subset. The model performs portfolio math through digit prediction, accumulating errors across every multiplication. It cannot compute a correlation matrix. It generates approximate sector percentages. It produces prose the analyst must verify. Across 5 turns: roughly 15,000 tokens for approximate partial analysis.

VDR-LLM-Prolog: Docker environment with network grant for market data API.

B391 `file_read` on `portfolio.csv`. B248 `parse_csv`. B228 `dict_from_pairs` converts each row to a dict. For each ticker, B424 `network_fetch` for current price. B254 `vdr_from_decimal_string` converts each price to exact VDR.

For each position: B003 `vdr_mul` of quantity by price. B047 `vdr_sum` across all 500 gives exact portfolio value. 500 integer multiplications, one integer sum. Zero accumulated error.

B208 `list_group_by` on sector. B047 `vdr_sum` within groups. B004 `vdr_div` by total gives exact sector allocation fractions. B202 `list_sort` by allocation.

Cost basis: B002 `vdr_sub` of current minus cost per position. B004 `vdr_div` by cost gives exact return percentage. B202 `list_sort` by return.

Correlation: B424 `network_fetch` for 90-day price histories. B082 `vdr_vec_dot` and B083 `vdr_vec_norm_sq` compute exact correlation coefficients. B085 `vdr_mat_new` builds the matrix. B093 `vdr_mat_det` verifies positive definiteness.

VaR: B202 `list_sort` on daily returns. B105 `vdr_stat_percentile` at [5, 100, 0] gives exact 95 percent VaR. B003 `vdr_mul` by portfolio value gives dollar VaR.

B247 `format_csv` exports results. B392 `file_write`.

The language model reads structured results and writes narrative commentary: roughly 300 tokens.

Token accounting — conventional: roughly 15,000 tokens for partial approximate analysis. VDR-LLM-Prolog: roughly 100 planning, 200 command tokens, 300 prose. Total: roughly 600. Reduction: 96 percent. Complete and exact versus partial and approximate.

7.6 Customer Support Knowledge Base

A company has 2,000 support articles. A support agent queries about a customer issue.

Conventional: the model has no access to the knowledge base. It generates an answer from training data that may be outdated or reference the wrong product version. The agent verifies manually. Each query: roughly 500 tokens of unreliable output.

VDR-LLM-Prolog: setup phase runs once. B404 `filesystem_glob` on articles directory. B391 `file_read` on each of 2,000 files. Markdown grammar extracts title, product version, error codes, problem description, solution steps, and related references as typed facts per article KB node. B363 `connection_add` links related articles. B342 `bitset_set` flags version coverage per article. Setup cost: roughly 100 language model tokens for planning, the rest is primitive calls.

Agent query: “customer on version 3.2 seeing error code 4017 after upgrade.”

B254 `vdr_from_decimal_string` converts version 3.2 to exact fraction [32, 10, 0]. B243 `to_number` converts 4017 to exact integer. B380 `kb_query_across` on predicate `error_code` with argument 4017 returns matching article IDs. B198 `list_filter` on version compatibility by `bitset` test for version 3.2. B367 `connection_graph` pulls related articles.

The language model reads matched articles’ content from KB addresses — roughly 200 tokens. Writes a response tailored to version 3.2: roughly 100 tokens. Grammar provides response structure.

Token accounting — conventional: roughly 500 tokens per query, unreliable. VDR-LLM-Prolog: roughly 150 per query, verified against source documentation. Reduction: 70 percent, plus the qualitative shift from unreliable training-data recall to exact KB query with provenance.

7.7 Academic Grading

A professor has 150 student essays to grade against a rubric.

Conventional: the model grades 3 essays before context fills. Rubric application drifts — the model is stricter on later essays because prior grading reasoning biases attention. It cannot compute class statistics. It cannot export a gradebook. The professor gets inconsistent grades with no structured data. Across many sessions: roughly 200,000 tokens for partial inconsistent grading.

VDR-LLM-Prolog: Docker environment with filesystem grant.

B391 file_read on rubric.json. B246 parse_json. Rubric stored as a dict. B404 filesystem_glob on submissions. 150 files. The language model writes a docx extraction script once — roughly 30 tokens. B410 execute_python runs it on each file.

For each essay: the parsed text goes into working data. The language model reads the essay and rubric criteria — this is the core judgment, roughly 300 tokens per essay. It produces scores per criterion. B254 vdr_from_decimal_string converts each score. B376 kb_assert stores fact(score, student_047, criterion_clarity, 8). B047 vdr_sum gives total. B298 counter_inc on essays_graded.

Grading is consistent because the rubric is a fixed struct the language model reads fresh from the KB for each essay. There is no context window accumulation of prior grading decisions biasing subsequent grades. Each essay is evaluated in a clean state against the same exact rubric.

After 150 essays: B378 kb_query collects all scores per criterion. B101 vdr_stat_mean, B102 vdr_stat_variance, B103 vdr_stat_median, B105 vdr_stat_percentile. All exact.

Individual feedback: for each student, B229 dict_get pulls scores. Comparison primitives compute class-relative position. Grammar template provides feedback structure. The language model fills personalized comment slots: roughly 80 tokens per student.

B247 format_csv exports the gradebook. B392 file_write.

Token accounting — conventional: roughly 200,000 tokens for partial inconsistent grading. VDR-LLM-Prolog: roughly 30 for the script, 45,000 reading and grading essays (300 per essay), 12,000 for feedback comments (80 per student), 200 command tokens. Total: roughly 57,230. Reduction: 71.4 percent. But the reduction understates the real difference: the 57,230 tokens are almost entirely judgment work — reading essays and writing feedback. The infrastructure cost is near zero. In the conventional case, a substantial fraction of the 200,000 tokens is re-reading prior essays, recapping rubric criteria, reconstructing grade distributions, and formatting output.

8. Accuracy by Construction

Every token the language model does not generate is a token that cannot be wrong. This is not a statistical improvement in accuracy. It is a structural elimination of error classes.

Six categories of error that occur in conventional language model output are impossible in VDR-LLM-Prolog output.

Arithmetic errors occur when a conventional language model generates digits through token prediction. The model predicts each digit as the most likely next token given the sequence so far. It has no mechanism to verify that 347 times 29 is 10,063 — it generates “10,063” because its training data associates multiplication expressions with their results, and the association is statistical, not computational. The error rate on multi-step arithmetic through token prediction is substantial and compounds with chain length. In VDR-LLM-Prolog, every arithmetic result comes from exact integer operations — B001 through B008 for closed VDR arithmetic, B139 through B159 for integer fast path, B076 through B100 for linear algebra. The operations are integer add, integer multiply, and integer divmod. Given correct inputs, the outputs are correct by the properties of integer arithmetic. The language model never generates a digit that represents a computed value.

State errors occur when a conventional language model loses track of established facts over conversation length. The model’s only state mechanism is the context window. As the window fills, attention distributes across more tokens, and the probability of correctly retrieving a specific fact decreases. In a 20-turn conversation, facts established in turn 3 may be misremembered or contradicted in turn 18 because the relevant tokens from turn 3 are competing with 50,000 other tokens for attention weight. In VDR-LLM-Prolog, every fact is stored at an integer address in a knowledge base. `fact(age, bob, 32)` asserted in turn 3 is at the same integer address in turn 18. B378 `kb_query` retrieves it exactly. There is no degradation with conversation length because retrieval is array indexing, not attention.

Formatting errors occur when a conventional language model generates structural tokens incorrectly — mismatched braces, misaligned columns, dropped delimiters, inconsistent indentation. Each structural token is predicted from the same vocabulary as content tokens, at the same computational cost, with the same error probability. In VDR-LLM-Prolog, grammar templates provide all structural tokens. The grammar knows the structure of its output format because the structure is declared in the grammar definition. A pipe-delimited table grammar produces correctly aligned pipes for every row because pipe placement is template logic, not token prediction.

Retrieval errors occur when a conventional language model recalls a fact incorrectly from training data. The model has no mechanism to distinguish accurate recall from fabrication — both are token sequences generated by the same prediction process. In VDR-LLM-Prolog, data retrieval is knowledge base query by integer address. The data at the address is the data that was stored there, with provenance recording who stored it, when, and from what source. Retrieval cannot produce data that was not stored because the mechanism is array indexing, not generation.

Deduction errors occur when a conventional language model produces invalid inferences in a reasoning chain. The model writes out each step as generated text, and any step can be a non-sequitur that reads plausibly but does not follow logically from the premises. In VDR-LLM-Prolog, deduction is Prolog evaluation. A rule fires only when its body goals are all satisfied by facts in scope. The derivation chain is structural — each step is a

unification against stored facts, not a generated text sequence. Invalid deductions cannot occur because the Prolog engine’s search procedure only returns results that unify.

Confidence errors occur when a conventional language model expresses uncertainty through hedging language — “approximately,” “likely,” “it appears that” — without any computational basis for the degree of uncertainty. The hedging is stylistic, trained from patterns in text where humans used similar language. In VDR-LLM-Prolog, confidence is an exact VDR fraction computed from declared propagation rules. When the system reports 90/100 confidence, that number was produced by B003 `vdr_mul` applying the multiple-independent-sources formula to the source confidences of the evidence. The computation is auditable. The propagation rules are declared Prolog facts. The confidence means something specific and verifiable.

The remaining error surface is language model judgment: intent recognition, step selection, prose generation. These are the tasks where the language model’s error rate is lowest because they are the tasks most aligned with what token prediction actually does — matching patterns in natural language and generating coherent text. By removing every other error source, the system’s overall error rate converges to the language model’s error rate on its strongest tasks only.

9. The Attention Reduction

Attention in a conventional transformer serves multiple functions simultaneously: determining what information is relevant to the current query, retrieving facts from earlier positions in the context, tracking conversational state, mapping relationships between entities, and weighting the importance of different pieces of information. Softmax normalization collapses all of these functions into a probability distribution over positions, enabling the model to perform weighted retrieval from its own context.

In VDR-LLM-Prolog, each of these functions is handled by a dedicated mechanism before the language model’s attention engages.

Relevance determination is handled by lexical scoping. The knowledge base tree’s scope chain makes irrelevant data structurally unreachable. When the active topic is `root.sessions.incident_2026_05_16`, knowledge bases outside that subtree and its ancestors are not deprioritized or ranked lower — they are invisible to queries. The language model’s attention does not need to determine what is relevant. The scope has already determined it.

Fact retrieval is handled by integer-addressed knowledge base query. B378 `kb_query` with a predicate and arguments returns matching facts from the scope chain. The retrieval is two integers — knowledge base ID and slot — followed by array indexing. The language model’s attention does not need to search for facts across positions in the context. The facts are at known addresses.

State tracking is handled by persistent knowledge base fields and bounded data primitives. Counter values, stack contents, LRU entries, ring buffer snapshots, and bitset states are

all at integer addresses, updated by primitive operations, and readable by primitive calls. The language model’s attention does not need to reconstruct state from prior conversation tokens.

Relationship mapping is handled by typed connections and Prolog rules. B366 `connection_list` and B367 `connection_graph` provide explicit typed relationships between knowledge bases. Prolog rules encode logical relationships between facts. The language model’s attention does not need to infer relationships from co-occurrence patterns in the token stream.

Importance weighting is handled by exact confidence fractions. Each finding carries a computed confidence from declared propagation rules. The language model’s attention does not need to learn which sources are more reliable — the confidence fractions encode this explicitly.

With all five functions handled structurally, the language model’s attention operates on pre-structured, pre-scoped, pre-typed, pre-weighted input. The input is a compact assembly of typed fields, not a raw token sequence. The attention’s task is reduced to the language model’s core competence: recognizing patterns in structured information and selecting appropriate next actions.

This reduced attention workload has implications for what attention mechanism the system requires. VDR-14 Appendix E specifies five attention mechanisms and four softmax variants. The full transcendental softmax (SM1, using Taylor-expanded exponentials) is available through Q335 functional remainders and produces attention weights summing to exactly one. But the reduced workload means simpler mechanisms may suffice.

The rational surrogate (SM2) computes attention weights as squared shifted inputs divided by the sum of all squared shifted inputs. It uses zero transcendentals — pure integer arithmetic. It produces exact sum-to-one normalization, preserves monotonicity, and yields exactly equal outputs for equal logits. Its denominator growth is minimal. For attention over pre-structured input where the discrimination task is simpler, the surrogate’s approximation of the exponential curve shape is likely sufficient.

Linear attention (AT4) eliminates softmax entirely, computing scores as kernel function products of queries and keys. If the kernel function is polynomial or ReLU — both exactly rational — the attention mechanism is fully VDR-native with no normalization step and no transcendentals. For the reduced workload of selecting among pre-typed, pre-scoped candidates, linear attention may be adequate.

The choice of attention mechanism becomes a practical decision based on the complexity of the judgment task, not an architectural constraint. The system can support full transcendental softmax for tasks that need it and simpler mechanisms for tasks that do not. The Q335 transcendental capability exists for domains that genuinely require it — physics, signal processing, quantum mechanics — not because the attention mechanism demands it.

10. Grammar as Token Eliminator

Grammar-directed generation, specified in VDR-12, has a direct token economics interpretation. Every structural token that grammar provides is a token the language model does not generate. The savings are quantifiable per output format.

In JSON output, structural tokens include opening and closing braces, opening and closing brackets, colons, commas, quotation marks around keys, quotation marks around string values, and whitespace for formatting. Across typical JSON objects, these structural tokens constitute roughly 55 percent of the total token count. A 200-token JSON response contains roughly 110 structural tokens and 90 content tokens. With grammar, the language model generates 90 tokens. The 110 structural tokens are provided by the grammar template at zero language model cost with guaranteed syntactic validity. The savings are 55 percent per JSON response with the additional benefit that malformed JSON — mismatched braces, dropped commas, unclosed strings — is structurally impossible.

In markdown table output, structural tokens include pipe characters, header separators, alignment indicators, and newlines between rows. These constitute roughly 60 percent of the total token count. A 20-row table that would cost 500 tokens as generated output costs roughly 200 tokens of language model generation for cell contents, with grammar providing the remaining 300 structural tokens. The savings are 60 percent per table, with guaranteed column alignment and consistent formatting.

In code output, structural tokens include braces, parentheses, semicolons, indentation, keywords, and syntax elements dictated by the language grammar. These constitute roughly 40 percent of the token count, depending on the language and the ratio of boilerplate to logic. A 100-token code snippet contains roughly 40 structural tokens. With grammar, the language model generates 60 tokens of logic and grammar provides the 40 tokens of syntax. The savings are 40 percent per code block, with guaranteed syntactic validity.

In structured prose output — reports, analyses, summaries with headers and sections — structural tokens include section headers, numbering, formatting markers, and template phrases. These constitute roughly 25 percent of the output for moderately structured prose. Grammar provides the structure; the language model fills the content.

The savings compound with output length. A 2,000-token report in JSON format saves roughly 1,100 tokens. A 5,000-token analysis with tables, code blocks, and structured prose saves roughly 2,500 tokens. Each saved token is not merely cheaper — it is structurally guaranteed correct, which eliminates a class of errors that increases with output length in conventional systems.

Grammar also operates on input. When structured input arrives — a table pasted by the user, a JSON payload from an API, a CSV export, a code block — grammar recognizes the structural tokens and extracts typed fields into knowledge base facts. The language model receives parsed, typed fields rather than raw text to interpret through attention. A 500-token markdown table consumed through attention becomes a list of 20 typed dicts consumed as structured data. The attention savings on input mirror the generation savings on output.

The bidirectional property of grammar means that the same grammar handles both directions. A grammar that produces pipe-delimited table output also parses pipe-delimited table input. A grammar that formats a comparison report also reads one. This means grammars are defined once and amortized over every use in both directions. The definition cost is roughly 10 to 30 language model tokens to assert the grammar rule into a knowledge base. Every subsequent use — every table rendered, every table parsed, every report formatted, every report read — costs zero language model tokens for the structural component.

The vocabulary constraint effect adds another dimension of savings. When a grammar declares that a slot accepts a categorical value from an enumeration with four legal values, the generation process can constrain the softmax to those four values instead of the full vocabulary. This is not a prompting hint that the model might ignore — it is a structural constraint derived from the grammar type declaration. The softmax over four candidates is computationally trivial compared to a softmax over 50,000 or more vocabulary entries. When a slot accepts an identifier from a knowledge base containing 200 named concepts, the softmax covers 200 candidates. The constraint reduces both the computation per token and the error probability per token simultaneously.

11. The Disposable Clone Advantage

Conversation drift is a recognized problem in conventional language models. As conversations extend, the model’s output quality degrades. Responses become less coherent, less accurate, and less focused. The degradation has multiple causes: attention dilution across growing context, accumulated noise from prior generation errors influencing subsequent tokens, and the lack of any mechanism to verify that the model’s current state is consistent with established facts.

In token economics terms, drift manifests as increasing infrastructure token cost per turn. The model spends more tokens on state reconstruction as the conversation grows and prior state becomes harder to locate in the context. It spends more tokens on hedging as its confidence in its own prior output decreases. It spends more tokens on correction when it notices inconsistencies with prior turns. The ratio of infrastructure tokens to judgment tokens worsens with every turn.

The disposable clone pattern specified in VDR-8 eliminates drift as a cost factor. The mechanism is straightforward. First, a session is built to a verified stable state: knowledge bases populated, constraints active, grammars loaded, operational grants issued. Second, a snapshot captures all live state atomically — typically 10 to 500 kilobytes. Third, disposable worker clones are spawned from that snapshot. Fourth, each clone is monitored against drift constraints: maximum session turns below a declared threshold, context saturation below a declared percentage, denominator growth below a declared budget, error rate below a declared percentage. Fifth, when any constraint fires, the drifting clone is killed and a fresh clone spawns from the same frozen snapshot.

The token economics of this pattern are significant. The language model inside a clone

always operates within its optimal range — early in a conversation, with low context saturation, before drift has begun. The infrastructure token overhead per turn stays at the level of turn 1 throughout the entire session, because any clone that would have degraded has been replaced. Work committed to persistent knowledge bases — facts asserted, rules established, connections made, findings recorded — survives clone death. The next clone starts with the clean snapshot state plus all accumulated persistent knowledge.

Over a 100-turn interaction (spanning multiple clone lifetimes), the conventional model's per-turn cost has escalated dramatically — each turn re-reads more history, generates more recapping, produces more hedging. The VDR-LLM-Prolog system's per-turn cost is constant because each clone operates within its first N turns, where N is the drift constraint threshold.

The compounding effect is that the system produces more accurate output with each successive clone. Clone 1 contributes facts A through D to the persistent knowledge base. Clone 2 starts fresh with clean state but has access to facts A through D at integer addresses. It contributes facts E through G. Clone 3 starts fresh with facts A through G available. The knowledge base grows monotonically while the language model stays permanently fresh. The conventional model has no equivalent — it cannot selectively preserve findings while discarding degradation, because both are interleaved in the same context window.

12. Cumulative Token Economics

The preceding sections analyze token economics per prompt and per conversation. The full economic picture emerges at workload scale — the total language model cost across a realistic day of work in a given domain.

Model a working day for an SRE team using language model assistance. Over 8 hours, the team processes 5 incidents of varying severity, conducts 3 postmortems for earlier incidents, reviews 10 monitoring dashboards for anomalies, and handles 20 ad-hoc queries about system behavior.

Under a conventional language model, each incident investigation averages 20 turns at roughly 3,000 tokens per turn, scaling with conversation length. Five incidents: roughly 5 times 20 times 3,000 base tokens, plus the quadratic attention scaling, totaling approximately 500,000 language model tokens. Each postmortem requires the SRE to reconstruct the investigation from memory, paste evidence, and iterate: roughly 10 turns at 4,000 tokens each for 3 postmortems, totaling approximately 120,000 tokens. Dashboard reviews are quick queries but require pasting screenshot descriptions or metric summaries: roughly 1,000 tokens each for 10 reviews, totaling approximately 10,000 tokens. Ad-hoc queries average 500 tokens each for 20 queries, totaling approximately 10,000 tokens. Day total: approximately 640,000 language model tokens. Quality degrades through each long investigation. Postmortem accuracy depends on human memory. No persistent state carries between incidents.

Under VDR-LLM-Prolog, each incident investigation averages 20 turns at roughly 210

language model tokens per turn (flat, no scaling with conversation length). Five incidents: roughly 5 times 20 times 210, totaling approximately 21,000 tokens. Each postmortem generates from the incident knowledge base that already exists — roughly 500 tokens each for 3 postmortems, totaling approximately 1,500 tokens. Dashboard reviews connect directly to prometheus through network primitives — roughly 150 tokens each for 10 reviews, totaling approximately 1,500 tokens. Ad-hoc queries resolve through knowledge base lookups and primitive calls — roughly 100 tokens each for 20 queries, totaling approximately 2,000 tokens. Day total: approximately 26,000 language model tokens. Quality improves through each investigation as knowledge accumulates. Postmortems are generated from exact structured data. Investigation findings persist across incidents — if incident 3 involves the same service as incident 1, all of incident 1’s findings are available at integer addresses.

The ratio is 640,000 to 26,000: approximately 24.6 to 1. But this understates the advantage because the 640,000 conventional tokens produce degrading-quality output with no persistent state, while the 26,000 VDR-LLM-Prolog tokens produce constant-quality output with accumulating persistent knowledge.

The cost per useful conclusion is the more meaningful metric. A useful conclusion is one that does not need correction, does not need verification against source data, and does not need to be re-derived in a subsequent conversation because the prior conversation’s state was lost. Under the conventional model, a substantial fraction of conclusions need correction (arithmetic errors, state confusion, retrieval errors) or re-derivation (no persistent state). Under VDR-LLM-Prolog, conclusions are exact, provenance-tagged, and persistent. The cost per useful conclusion is not merely 24.6 times lower on raw tokens — it is further reduced by the elimination of rework.

The persistent knowledge base also means that day 2 starts where day 1 ended. The conventional model starts from zero on every conversation. VDR-LLM-Prolog starts with every fact, rule, connection, grammar, and finding from all prior work at integer addresses. The system’s effective capability increases over operational lifetime while its per-prompt token cost remains flat.

Appendix A: Token Cost Comparison Tables

A.1: Per Use Case Summary

Use Case	Conventional Tokens	VDR-LLM-Prolog Tokens	Reduction	Conventional Accuracy	VDR Accuracy
SRE incident (5 turns)	15,000	210	98.6%	Degrading with context length	Constant; exact metrics

Use Case	Conventional Tokens	VDR-LLM-Prolog Tokens	Reduction	Conventional Accuracy	VDR Accuracy
Legal contract review (10 turns)	30,000	1,130	96.2%	Loses cross-references	Full cross-reference via KB
Medical synthesis (40 papers)	80,000	4,740	94.1%	Misses distant contradictions	Exact stats; all contradictions detected
Codebase migration (200 files)	100,000 (partial)	6,700 (complete)	93.3%	Partial with regressions	Complete with test verification
Financial portfolio (500 positions)	15,000 (partial)	600	96.0%	Approximate arithmetic	Exact arithmetic; zero drift
Customer support (per query)	500	150	70.0%	Training data recall	Indexed KB query with provenance
Academic grading (150 essays)	200,000 (partial)	57,230 (complete)	71.4%	Inconsistent; rubric drift	Consistent; clean rubric per essay

A.2: Token Category Breakdown — SRE Incident

Category	Conventional	VDR-LLM-Prolog	Notes
Context re-reading (attention)	60,000+ across 5 turns	0	State in KB, not context window
State reconstruction	2,250	0	KB query by integer address

Category	Conventional	VDR-LLM-Prolog	Notes
Input parsing	Implicit in attention	0	Cleanup and classification primitives
Computation	3,000	0	Exact arithmetic primitives
Deduction	2,250	0	Prolog evaluation
Formatting	3,000	0	Grammar templates
Hedging	1,500	0	Exact confidence fractions
Command tokens	0	80	Primitive invocations
Judgment (assessment)	1,500	50	Cleaner input, same task
Prose output	1,500	80	Grammar handles structure
Total generated	15,000	210	—

A.3: Token Category Breakdown — Financial Portfolio

Category	Conventional	VDR-LLM-Prolog	Notes
Context re-reading	30,000+ across 5 turns	0	Positions in KB
Data parsing	3,000	0	CSV parse primitive
Arithmetic (500 multiplications)	5,000	0	Exact VDR primitives
Aggregation (sector sums)	2,000	0	Sum and group-by primitives
Correlation computation	Cannot perform	0	Linear algebra primitives
Formatting	3,000	0	Grammar templates
Hedging	1,000	0	Exact confidence

Category	Conventional	VDR-LLM-Prolog	Notes
Command tokens	0	200	Primitive invocations
Planning	500	100	Structured input
Prose commentary	500	300	Grammar handles data display
Total generated	15,000	600	—

A.4: Token Category Breakdown — Academic Grading

Category	Conventional	VDR-LLM-Prolog	Notes
Context re-reading	400,000+ across sessions	0	Each essay reads from fresh KB state
Rubric re-reading	30,000	0	Fixed struct read per essay
Essay reading and scoring	45,000	45,000	Irreducible judgment work
Statistical computation	Cannot perform	0	Exact stat primitives
Feedback generation	20,000	12,000	Grammar handles structure
Formatting	40,000	0	Grammar templates
Consistency overhead	65,000	0	No drift; clean state per essay
Command tokens	0	200	Primitive invocations
Script authoring	0	30	Written once
Total generated	200,000	57,230	—

Appendix B: Crossover Analysis

B.1: Single Turn Crossover

Saved To-kens	Primitive Ops	Equivalent Token Budget for Primitives	Float Ops per LLM Token	Q335 Ops per Primitive	Required Q335 Slowdown for Breakeven
2,790	500	$2,790 \times C$	$\sim 10^6$ per token	$\sim 10^3$ per op	$\sim 5,580 \times$
2,790	200	$2,790 \times C$	$\sim 10^6$ per token	$\sim 10^3$ per op	$\sim 13,950 \times$
2,790	50	$2,790 \times C$	$\sim 10^6$ per token	$\sim 10^3$ per op	$\sim 55,800 \times$

C = cost of one language model forward pass token. Q335 would need to be thousands to tens of thousands of times slower per operation to break even.

B.2: Multi-Turn Crossover (SRE Investigation)

Turn	Conventional Cumulative Tokens	VDR Cumulative LLM Tokens	VDR Cumulative Primitive Ops	Margin (tokens saved)	Required Q335 Slowdown
1	3,000	210	500	2,790	$\sim 5,580 \times$
5	18,000	1,050	2,500	16,950	$\sim 6,780 \times$
10	45,000	2,100	5,000	42,900	$\sim 8,580 \times$
20	120,000	4,200	10,000	115,800	$\sim 11,580 \times$
50	500,000	10,500	25,000	489,500	$\sim 19,580 \times$

The required slowdown grows with conversation length because conventional token cost scales quadratically (attention re-reading) while primitive cost scales linearly.

B.3: Workday Crossover (SRE Team)

Workload	Conventional Tokens	VDR LLM Tokens	VDR Primitive Ops	Margin	Required Slowdown
1 incident (20 turns)	120,000	4,200	10,000	115,800	$\sim 11,580 \times$
5 incidents + post-mortems	620,000	22,500	50,000	597,500	$\sim 11,950 \times$

Workload	Conventional Tokens	VDR LLM Tokens	VDR Primitive Ops	Margin	Required Slowdown
Full day (all tasks)	640,000	26,000	60,000	614,000	$\sim 10,233 \times$
Full week	3,200,000	130,000	300,000	3,070,000	$\sim 10,233 \times$

The margin stabilizes at roughly $10,000 \times$ because the ratio of token savings to primitive operations converges. Q335 is nowhere near 10,000 times slower than float. The token economics structurally dominate.

Appendix C: Builtin Call Patterns

C.1: Data Ingestion Pattern

Used in: all use cases. Fetches external data, parses it, converts values to exact VDR, stores in KB.

Step	Builtin	Tokens	Purpose
Fetch	B391 file read or B424 network fetch	~ 8	Raw bytes enter system
Parse	B246 parse json or B248 parse csv	~ 8	Typed structure extracted
Convert	B254 vdr from decimal string (per value)	~ 8 each	Exact VDR at conversion boundary
Store	B376 kb assert (per fact)	~ 8 each	Provenance-tagged KB storage

Total per data source: roughly 32 base tokens plus 16 per extracted value.

C.2: Filter-Sort-Aggregate Pattern

Used in: SRE metrics, portfolio analysis, support KB queries, grading statistics.

Step	Builtin	Tokens	Purpose
Filter	B198 list filter	~ 8	Subset by predicate
Sort	B202 list sort	~ 8	Order by key
Take	B187 list take	~ 8	Top N results
Aggregate	B047 vdr sum or B049 vdr mean	~ 8	Exact computation
Store	B233 dict set	~ 8	Results to working data

Step	Builtin	Tokens	Purpose
------	---------	--------	---------

Total: roughly 40 tokens for the complete filter-sort-aggregate pipeline on any dataset size.

C.3: Hypothesis-Test Pattern

Used in: SRE investigation, medical contradiction detection, decision analysis.

Step	Builtin	Tokens	Purpose
Formalize rule	B376 kb assert (Prolog rule)	~20	LLM writes the rule
Evaluate	B378 kb query	~8	Prolog engine evaluates
Compute confidence	B003 vdr mul, B002 vdr sub	~16	Propagation formula
Store finding	B376 kb assert	~8	Provenance-tagged result
Update counters	B298 counter inc	~8	Budget tracking

Total: roughly 60 tokens per hypothesis tested.

C.4: Correlation-Join Pattern

Used in: SRE deployment correlation, medical contradiction detection, supply chain trend analysis.

Step	Builtin	Tokens	Purpose
Fetch set A	B378 kb query	~8	First data collection
Fetch set B	B378 kb query	~8	Second data collection
Join	B198 list filter + B196 list contains	~16	Match on shared key
Compare	B017 vdr compare per pair	~8 each	Exact comparison
Group	B208 list group by	~8	Organize results
Sort	B202 list sort	~8	Rank by magnitude

Total: roughly 56 base tokens plus 8 per comparison.

C.5: Document Processing Pattern

Used in: legal contract review, document summarization, academic grading.

Step	Builtin	Tokens	Purpose
Read file	B391 file read	~8	Raw bytes
Extract structure	B410 execute python (written once)	~30 first time, ~8 reuse	Parsing script
Parse output	B246 parse json	~8	Typed dict tree
Enumerate sections	B236 dict keys	~8	Section list
Pull section	B229 dict get per section	~8 each	Section content
LLM reads and judges	—	Varies	Irreducible judgment
Store result	B233 dict set	~8 each	Per-section results
Export	B247 format csv + B392 file write	~16	Final output

Total infrastructure: roughly 80 base tokens plus 16 per section. LLM tokens scale only with judgment work.

Appendix D: Grammar Savings Quantification

D.1: Structural Token Percentage by Format

Output Format	Total Tokens (conventional)	Structural Tokens	Content Tokens	Structural Percentage	Grammar Savings
JSON object (20 fields)	200	110	90	55%	110 tokens
Markdown table (20 rows, 5 cols)	500	300	200	60%	300 tokens
CSV (50 rows, 8 cols)	400	200	200	50%	200 tokens
Code block (Python, 30 lines)	300	120	180	40%	120 tokens

Output Format	Total Tokens (conventional)	Structural Tokens	Content Tokens	Structural Percentage	Grammar Savings
Structured report (5 sections)	2,000	500	1,500	25%	500 tokens
Incident report template	800	400	400	50%	400 tokens
Comparison table (2 items, 10 criteria)	350	210	140	60%	210 tokens

D.2: Error Elimination by Format

Output Format	Common Conventional Errors	Error Probability (conventional)	Error Probability (grammar)
JSON	Mismatched braces, missing commas, unclosed strings	~2-5% per response	0%
Markdown table	Misaligned columns, missing pipes, inconsistent headers	~5-10% per table	0%
CSV	Unescaped commas, inconsistent quoting, wrong delimiter	~3-7% per export	0%
Code syntax	Missing semicolons, mismatched brackets, wrong indentation	~5-15% per block	0% for structure
Numbered lists	Misnumbered items, inconsistent formatting	~3-8% per list	0%

D.3: Vocabulary Constraint Effect

Slot Type	Full Vocabulary Size	Constrained Vocabulary Size	Softmax Reduction	Error Rate Reduction
Boolean field	50,000+	2 (yes/no)	25,000 $\times$	~99.99%
Enum (4 values)	50,000+	4	12,500 $\times$	~99.99%
KB identifier (200 concepts)	50,000+	200	250 $\times$	~99.6%
Numeric (integer, 3 digits)	50,000+	1,000	50 $\times$	~98%
Free text	50,000+	50,000+	1 $\times$	Baseline

Appendix E: Attention Mechanism Comparison Under Reduced Load

E.1: Workload per Function

Attention Function	Conventional Load	VDR-LLM-Prolog Load	Reduction Mechanism
Relevance determination	Full context window	Pre-scoped by KB tree	Lexical scoping
Fact retrieval	Search across all positions	Not needed	Integer-addressed KB query
State tracking	Implicit across history	Not needed	Persistent KB fields
Relationship mapping	Inferred from co-occurrence	Not needed	Typed connections
Importance weighting	Learned from training	Not needed	Exact confidence fractions
Intent recognition	User message + context	User message + structured summary	Pre-parsed, pre-typed input
Step selection	Full reasoning over context	Selection from known primitives	Low-entropy reference selection

E.2: Mechanism Suitability Under Reduced Load

Mechanism	Transcendental	VDR-Native	Denominator Growth	Suitable for Reduced Load	Suitable for Full Load
SM1 Taylor softmax	Yes	Via Q335 fn remainder	Moderate (factorial terms)	Yes (overqualified)	Yes
SM2 Ratio-nal surrogate	No	Fully native	Low (small powers)	Yes (well-matched)	Approximate shape
SM4 Padé approximant	No	Fully native	Low (polynomial)	Yes (well-matched)	Better than Taylor at same degree
AT4 Linear attention	No	Fully native if kernel is polynomial/ReLU	Minimal	Yes (simplest option)	May underperform on complex tasks
AT5 Surrogate-weighted	No	Fully native	Low	Yes	Yes for pre-structured input

E.3: Computational Cost per Attention Step

Mechanism	Ops per Query-Key Score	Ops per Normalization	Total per Head per Position	Notes
SM1 Taylor (depth 10)	d k multiplications	~10 multiply + add per logit	High	Full transcendental computation

Mechanism	Ops per Query-Key Score	Ops per Normalization	Total per Head per Position	Notes
SM2 Surrogate	d k multiplications	1 subtract + 1 square + 1 sum per logit	Low	Integer arithmetic only
AT4 Linear (ReLU kernel)	d k multiplications + ReLU	No normalization step	Lowest	No softmax at all

For pre-structured input with a small number of typed candidates, the difference between mechanisms is minimal. The choice is practical, not architectural.

Appendix F: Capability Boundary Cases

F.1: Data Volume Boundaries

Data Type	Size	Token Equivalent	Conventional LLM	VDR-LLM-Prolog	Mechanism
JSON metrics	1 MB	~300,000 tokens	Cannot load	Routine	B424 fetch + B246 parse
Document (docx)	10 MB	~2,500,000 tokens	Cannot load	Routine	B391 read + B410 script parse
CSV dataset	5 MB	~1,200,000 tokens	Cannot load	Routine	B391 read + B248 parse
Code repository	200 files	~500,000 tokens	Cannot hold simultaneously	Routine	B405 tree + B391 per-file
Knowledge base	2,000 articles	~5,000,000 tokens	Cannot search	Routine	B404 glob + indexed KB query
Portfolio	500 positions	~25,000 tokens	Overflows useful context	Routine	B248 parse + primitives

F.2: Computational Boundaries

Computation	Scale	Conventional LLM	VDR-LLM-Prolog	Mechanism
Arithmetic chain	500 multipli-cations	Accumulating errors	Exact	VDR primitives
Correlation matrix	500×500	Cannot compute	Exact	B082, B085 linear algebra
Statistical aggregation	10,000 values	Cannot hold + compute	Exact	B101-B105 stat primitives
Dependency graph	200 nodes	Cannot build	Exact	B285, B295 graph primitives
Time series trend	90 daily points $\times$ 50 series	Cannot process all	Exact	B070 discrete derivative
Cross-reference resolution	2,000 documents	Cannot hold	Constant-time	B380 kb query across

F.3: State Persistence Boundaries

Scenario	Conventional LLM	VDR-LLM-Prolog	Mechanism
20-turn investigation	Degrades; loses early findings	All findings at integer addresses	KB persistence
Cross-session continuity	Starts from zero	Full prior state available	Persistent KB tree
Multi-incident correlation	Cannot access prior incidents	Prior incident KBs queryable	Connection traversal
Team knowledge accumulation	Per-conversation only	Organization-wide KB tree	Hierarchical KB structure
Postmortem generation	Requires human reconstruction	Generated from incident KB	Provenance chain

Appendix G: Conversation Length Scaling Data

G.1: Token Cost Scaling

Turn	Conventional Generated	Conventional Attention	VDR Generated	VDR Primitive Ops
1	3,000	3,000	210	500
5	3,000	15,000	210	500
10	3,000	30,000	210	500
20	3,000	60,000	210	500
50	3,000	150,000	210	500
100	3,000	300,000	210	500

Conventional attention cost grows linearly per turn (all prior history re-read). VDR primitive cost is constant (state at integer addresses).

G.2: Cumulative Cost

Through Turn	Conventional Total (generated + attention)	VDR Total (LLM + primitive equivalent)	Ratio
1	6,000	260	23:1
5	60,000	1,300	46:1
10	195,000	2,600	75:1
20	690,000	5,200	133:1
50	3,900,000	13,000	300:1
100	15,300,000	26,000	588:1

The ratio grows continuously because conventional cost is quadratic (sum of linear attention growth) while VDR cost is linear (constant per turn).

G.3: Quality Scaling

Turn	Conventional Accuracy Trend	VDR Accuracy Trend	Conventional State Coverage	VDR State Coverage
1	Baseline	Baseline	Complete (small context)	Complete (all in KB)
5	Slight degradation	Improved (5 turns of findings)	Good	Complete
10	Noticeable degradation	Further improved	Gaps appearing	Complete

Turn	Conventional Accuracy Trend	VDR Accuracy Trend	Conventional State Coverage	VDR State Coverage
20	Significant degradation	Full investigation accumulated	Significant gaps	Complete
50	Severe degradation	Mature knowledge base	Majority lost	Complete
100	Unreliable	Comprehensive knowledge base	Near-total loss	Complete

Conventional accuracy degrades because attention dilutes and prior facts compete with noise. VDR accuracy improves because each turn contributes exact, provenance-tagged facts to persistent storage. The curves move in opposite directions.

Appendix H: Primitive Cost Classification

H.1: Cost Tiers by Operation Type

Tier	Integer Ops per Call	Examples	Builtin IDs	Percentage of 448
Trivial (1-3 ops)	1-3	Counter inc, lock check, bitset test, dict get, list nth	B298-B350, B229, B186	~28%
Light (4-10 ops)	4-10	VDR add, compare, string contains, list filter step, kb query match	B001, B017-B026, B168, B198, B378	~35%

Tier	Integer Ops per Call	Examples	Builtin IDs	Percentage of 448
Medium (10-100 ops)	10-100	VDR multiply (1 mul + 1 divmod), sort (n log n comparisons), gcd	B003, B202, B034	~25%
Heavy (100-10,000 ops)	100-10,000	Matrix multiply, determinant, softmax, discrete derivative	B089, B093, B115, B070	~10%
Intensive (10,000+ ops)	10,000+	Matrix inverse (large), fn sqrt deep, correlation matrix, DFT	B094, B062 depth>10, B454	~2%

H.2: Primitive Cost vs LLM Token Cost

Primitive	Integer Ops	Equivalent LLM Tokens (at 10^6 float ops per token)	Typical Use
B298 counter inc	1	0.000001	Budget tracking
B229 dict get	2	0.000002	Data retrieval
B342 bitset test	1	0.000001	Membership check
B001 vdr add (Q335)	3	0.000003	Exact addition
B017 vdr compare	4	0.000004	Exact comparison
B003 vdr mul (Q335)	~200	0.0002	Exact multiplication

Primitive	Integer Ops	Equivalent LLM Tokens (at 10^6 float ops per token)	Typical Use
B034 vdr gcd	~50	0.00005	Simplification
B047 vdr sum (500 items)	~1,500	0.0015	Portfolio total
B202 list sort (500 items)	~4,500	0.0045	Ranking
B093 vdr mat det (10×10)	~55,000	0.055	Determinant
B082 vdr vec dot (90 dim)	~18,000	0.018	Correlation coefficient
B115 vdr softmax (1000 logits, depth 10)	~100,000	0.1	Attention normalization

Every primitive in the table costs less than one LLM token equivalent. Most cost less than one thousandth of a token.

Appendix I: Data Primitive Memory Footprint

I.1: Per-Primitive Memory Cost

Primitive	Typical Capacity	Bytes per Entry	Typical Total Bytes	Notes
Counter	1 value	16 (i64 + bounds)	16	Min/max bounds stored
Lock	1 flag	24 (bool + holder + timestamp)	24	Non-blocking
Queue	64 entries	32 per entry	2,048	Bounded FIFO
Stack	32 entries	32 per entry	1,024	Bounded LIFO
LRU Cache	128 entries	64 per entry (key + value + timestamp)	8,192	Evicts old-est

Primitive	Typical Capacity	Bytes per Entry	Typical Total Bytes	Notes
Ring Buffer	90 entries	32 per entry	2,880	Overwrites old-est Packed bits
Bitset	256 bits	32 bytes	32	

I.2: Incident Investigation Memory Budget

Primitive	Instance	Capacity	Bytes	Purpose
Counter	steps executed	1	16	Budget tracking
Counter	queries issued	1	16	Budget tracking
Counter	hypotheses tested	1	16	Budget tracking
Counter	steps since evidence	1	16	Stall detection
LRU	findings	128	8,192	Recent findings with provenance
LRU	sources	64	4,096	Data sources consulted
LRU	attempted steps	64	4,096	Failed approaches with reasons
Stack	investigation path	32	1,024	Backtracking support
Bitset	deps checked	256	32	Which dependencies investigated
Lock	investigating	1	24	Coordination flag
Ring Buffer	metrics	90	2,880	Rolling metric snapshots
Queue	remediation	32	1,024	Steps to execute
Total	—	—	21,432	~21 KB for full incident state

One incident investigation’s complete live state fits in 21 KB. A session snapshot capturing this is roughly 25 KB with overhead. Compare to a conventional LLM’s context window holding the same investigation: 60,000+ tokens at roughly 4 bytes per token is 240+ KB of raw token data, none of which is typed, indexed, or queryable.

I.3: Snapshot Size by Use Case

Use Case	Active KBs	Data Primitives	Snapshot Size	Conventional Context Equivalent
SRE incident	5	12	~25 KB	~240 KB (60k tokens)
Legal contract review	8	6	~15 KB	~120 KB (30k tokens)
Medical synthesis	42	8	~40 KB	~320 KB (80k tokens)

Use Case	Active KBs	Data Primitives	Snapshot Size	Conventional Context Equivalent
Codebase migration	205	10	~180 KB	~400 KB (100k tokens)
Financial portfolio	10	6	~50 KB	~60 KB (15k tokens)
Support KB (post-setup)	3	4	~8 KB	~2 KB (500 tokens per query)
Academic grading	155	8	~120 KB	~800 KB (200k tokens)

Snapshots are 3x to 7x smaller than the equivalent conventional context, and they contain typed queryable state rather than raw token sequences.

Appendix J: Command Token Entropy Analysis

J.1: Entropy per Token Category

Token Category	Vocabulary Size	Bits of Entropy per Token	Tokens per Unit	Total Bits per Unit
Primitive name	~300	~8.2	1-2	~12
KB path segment	~200 per scope level	~7.6	2-4	~23
Literal integer	Unbounded but typically <1000	~10	1	~10
Slot type tag	5 (cmd, query, assert, script, await)	~2.3	1	~2.3
Await flag	2	1	1	1
Command token total	—	—	~8	~48

J.2: Comparison to Free-Form Generation

Generation Mode	Vocabulary	Bits per Token	Tokens per Equivalent Action	Total Bits	Error Modes
Command token	~300 names + ~200 paths	~6 avg	~8	~48	Invalid name, wrong path
JSON function call	50,000+	~15.6	~30	~468	Syntax error, wrong key, type mismatch, malformed JSON
Free-form code generation	50,000+	~15.6	~50	~780	All syntax errors, logic errors, runtime errors
Natural language instruction	50,000+	~15.6	~100	~1,560	Ambiguity, misinterpretation, incompleteness

Command tokens are 10x lower entropy than JSON function calls and 16x lower than natural language. Lower entropy means lower error probability per token and fewer tokens per action.

J.3: Error Probability Estimation

Mode	Tokens Generated	Error Probability per Token	Probability of Error-Free Action
Command token (8 tokens)	8	~0.1% (constrained vocab)	~99.2%

Mode	Tokens Generated	Error Probability per Token	Probability of Error-Free Action
JSON func- tion call (30 to- kens)	30	~0.5% (syntax-sensitive)	~86.0%
Free- form code (50 to- kens)	50	~1% (logic + syntax)	~60.5%
Natural lan- guage rea- son- ing (100 to- kens)	100	~2% (semantic drift)	~13.3%

The probability of an error-free action drops dramatically with token count and entropy. Command tokens' combination of low count and low entropy per token gives the highest reliability.

Appendix K: Context Window Utilization

K.1: Conventional Context Allocation (SRE Investigation, Turn 20)

Content Type	Tokens	Percentage	Queryable	Typed	Provenance
System prompt	2,000	3.3%	No	No	N/A
Prior user messages (turns 1-19)	8,000	13.3%	Via attention only	No	No
Prior assistant responses (turns 1-19)	38,000	63.3%	Via attention only	No	No
Current user message	500	0.8%	Via attention only	No	No

Content Type	Tokens	Percentage	Queryable	Typed	Provenance
Pasted metric data (raw)	10,000	16.7%	Via attention only	No	No
Available for new generation	1,500	2.5%	N/A	N/A	N/A
Total context window	60,000	100%	—	—	—

63.3% of the context window is the model’s own prior output — state recaps, formatting, hedging, computation attempts. 16.7% is raw pasted data the model parses through attention. 2.5% is available for new work. The model operates in a self-congested channel.

K.2: VDR-LLM-Prolog Context Allocation (Same Investigation, Turn 20)

Content Type	Tokens	Percentage	Queryable	Typed	Provenance
System prompt + grammar defs	500	33.3%	By grammar reference	Yes	N/A
Structured state summary	200	13.3%	By KB address	Yes	Yes
Current user message (cleaned)	100	6.7%	By classification tag	Yes	N/A
Matched KB data (pulled for this step)	200	13.3%	By integer address	Yes	Yes
Available for judgment + generation	500	33.3%	N/A	N/A	N/A
Total LLM input	1,500	100%	—	—	—

33.3% of the input is available for new work versus 2.5% in the conventional case. The effective context capacity for judgment is 13x higher, at turn 20, while the total input size is 40x smaller.

K.3: Context Efficiency Ratio by Turn

Turn	Conventional Total Context	Conventional Available for Judgment	VDR Total Input	VDR Available for Judgment	Efficiency Ratio
1	5,000	2,000 (40%)	800	400 (50%)	1.25 ×
5	18,000	1,800 (10%)	1,000	450 (45%)	4.5 ×

Turn	Conventional Total Context	Conventional Available for Judgment	VDR Total Input	VDR Available for Judgment	Efficiency Ratio
10	33,000	1,500 (4.5%)	1,200	480 (40%)	$8.9 \times$
20	63,000	1,500 (2.4%)	1,500	500 (33%)	$13.8 \times$
50	153,000	Saturated (0%)	1,500	500 (33%)	∞

The conventional model saturates — its context window fills entirely with history and it can no longer accept new data. The VDR system never saturates because prior history is in KBs, not the context window.

Appendix L: Grammar Amortization

L.1: Grammar Definition Cost vs Reuse Savings

Grammar Type	Definition Cost (LLM tokens)	Structural Tokens Saved per Use	Uses to Break Even	Typical Uses per Session	Net Savings per Session
JSON object	15	110	1	10+	1,085+ tokens
Markdown table	12	30 per row	1	5+ tables (20 rows each)	2,988+ tokens
CSV export	10	20 per row	1	3+ exports	1,790+ tokens
Incident report	25	400	1	1+	375+ tokens
Feedback template	20	60	1	150 (per student)	8,980 tokens
API re-sponse format	15	80	1	100+ queries	7,985+ tokens
Postmortem template	20	350	1	Reused across incidents	330+ per use
Comparison table	11	210	1	3+	618+ tokens

Every grammar breaks even on first use. Subsequent uses are pure savings. Grammars persist in the KB tree and inherit through scope, so a grammar defined at an organizational level amortizes across every user, session, and conversation beneath it.

L.2: Inherited Grammar Cascade

Scope Level	Grammars Defined	Inherited By	Total Effective Grammars	Definition Cost (once)	Amortization Base
root.system	8 (core formats)	Everything	8	120 tokens	All sessions ever
root.org.acme	5 (company templates)	All Acme users	13	75 tokens	All Acme sessions
root.org.acme.sre	4 (SRE-specific)	SRE team	17	60 tokens	All SRE sessions
root.org.acme.sre.session N	2 (session-specific)	This session	19	30 tokens	This session

An SRE session has 19 effective grammars available. Only 2 were defined in the session. The other 17 were defined once at higher scopes and inherited at zero cost. Total definition cost across the full hierarchy: 285 tokens, paid once, reused indefinitely.

Appendix M: Prolog Rule Composition Economics

M.1: Composited Operation Token Cost

Composited Operation	Component Primitives	Tokens to Formalize Rule	Equivalent Conventional Tokens	Reuses per Session	Amortized Cost per Use
Steady-state entropy	B129 markov steady state + B114 prob entropy terms	25	200+ (manual computation)	5+	5 tokens
Weighted anomaly score	B049 vdr mean + B002 vdr sub + B006 vdr abs + B004 vdr div	30	150+	20+	1.5 tokens

Composed Operation	Component Primitives	Tokens to Formalize Rule	Equivalent Conventional Tokens	Reuses per Session	Amortized Cost per Use
SLA breach projection	B337 ring buffer read all + B070 discrete derivative + B003 vdr mul + B017 vdr compare	35	300+	50+	0.7 tokens
Cross-reference check	B378 kb query + B198 list filter + B366 connection list	20	100+	100+	0.2 tokens
Contradiction detection	B378 kb query + B023 vdr sign + B019 vdr less than	40	250+	40+	1 token
Migration point classifier	B163 string split + B198 list filter + B208 list group by	25	180+	200+	0.125 tokens
Rubric score normalizer	B047 vdr sum + B004 vdr div + B017 vdr compare	20	80+	150+	0.13 tokens

M.2: Rule Persistence and Scope

Assertion Scope	Lifetime	Available To	Token Cost	Use Count Potential
Session KB	Dies with session	This session only	Same	10-100

Assertion Scope	Lifetime	Available To	Token Cost	Use Count Potential
Project KB	Persistent	All project sessions	Same	100-1,000
Team KB	Persistent	All team members	Same	1,000-10,000
Organization KB	Persistent	All org members	Same	10,000+
root.system	Permanent	Entire system	Same	Unlimited

A 25-token Prolog rule asserted at the organization level and reused 10,000 times costs 0.0025 tokens per use. The same capability in a conventional LLM is re-derived from scratch every single time at full token cost.

Appendix N: Conversion Boundary Precision

N.1: External Value Types at Conversion Boundary

Source Type	Example Input	B254 Output	Exact	Max Error	KB Provenance Fact
Terminating decimal	"3.14"	[314, 100, 0]	Yes	0	source:decimal, error:0
Integer string	"4017"	[4017, 1, 0]	Yes	0	source:integer, error:0
Currency	"\$2,500,000.00"	[25000000, 100, 0]	Yes	0	source:currency, error:0
Percentage	"47.3%"	[473, 1000, 0]	Yes	0	source:percentage, error:0
Scientific notation	"2.78e-16"	[278, 10 ¹⁸ , 0]	Yes	0	source:scientific, error:0
Prometheus gauge	"0.847"	[847, 1000, 0]	Yes	0	source:prometheus, error:0
Float-origin metric	"3.1415926535897931592653589793"	[3141592653589793, 10 ¹⁵ , 0]	source precision	$\pm 5 \times 10^{-16}$	source:float64, error:5e-16
Repeating decimal	"0.333..."	[1, 3, 0]	If pattern known	0 if recognized	source:repeating, error:0

Every value entering the system has its conversion method, original representation, and maximum error recorded as a KB fact. The provenance chain never silently introduces

approximation.

N.2: Error Propagation from Conversion Boundary

Operation	Input Error	Output Error	Mechanism
VDR add of two exact values	0	0	Integer addition
VDR add of exact + float-origin	$0 + \varepsilon$	ε	Error bound inherited
VDR mul of two exact values	0	0	Integer mul + divmod
VDR mul of exact $\times$ float-origin	$0 \times \varepsilon$	Bounded by input $\varepsilon \times$ value	Propagation rule
Chain of 500 exact multiplications	0	0	No accumulation possible
Chain of 500 float-origin multiplications	$\varepsilon_1 \dots \varepsilon_{500}$	Bounded by product of $(1 + \varepsilon_i)$	Declared, auditable

The critical distinction: in a conventional system, every operation introduces new error silently. In VDR, operations on exact values produce exact results, and operations on float-origin values have bounded, tracked, declared error that never grows silently.

Appendix O: Grant Budget per Use Case

O.1: Minimum Grant Sets

Use Case	Grant Class	Allowed Operations	Location Constraint	Max Uses	Notes
SRE incident	network	fetch, post	prom.internal:9090/*100		Prometheus access
SRE incident	filesystem	write	/reports/*	5	Report export
Legal contract	filesystem	read	/uploads/*	10	Document ingestion
Legal contract	execute	python	sandboxed	5	docx parser
Medical synthesis	network	fetch	api.crossref.org/*	200	Paper meta-data

Use Case	Grant Class	Allowed Operations	Location Constraint	Max Uses	Notes
Codebase migration	filesystem	read, write	/project/*	1000	Full project access
Codebase migration	execute	python, pytest	sandboxed	500	Analysis and testing
Financial portfolio	filesystem	read	/data/*	10	Portfolio file
Financial portfolio	network	fetch	api.marketdata.com/*	600	500 tickers + history
Financial portfolio	filesystem	write	/reports/*	5	Export
Support KB setup	filesystem	read	/articles/*	2500	Bulk ingestion
Academic grading	filesystem	read, write	/submissions/, /grades/	500	Essays in, grades out
Academic grading	execute	python	sandboxed	5	docx parser

O.2: Grant Consumption Tracking

Use Case	Grants Issued	Grants Consumed	Remaining	Consumption Rate	Logged Facts Generated
SRE incident (5 turns)	105	23	82	21.9%	23
Legal contract (10 turns)	15	8	7	53.3%	8
Medical synthesis (40 papers)	200	82	118	41.0%	82

Use Case	Grants Issued	Grants Consumed	Remaining	Consumption Rate	Logged Facts Generated
Codebase migration (200 files)	1500	620	880	41.3%	620
Financial portfolio	615	510	105	82.9%	510
Support KB setup	2500	2003	497	80.1%	2003
Academic grading	505	305	200	60.4%	305

Every grant consumption is a logged KB fact. The entire operational history is queryable. A security audit can reconstruct exactly which operations were performed, on what resources, at what time, by which session.

Appendix P: Disposable Clone Lifecycle

P.1: Drift Constraint Thresholds

Constraint	Threshold	Measurement	Check Frequency	On Violation
max session turns	200	B304 counter get on turn counter	Every turn	Kill + respawn
context saturation	90%	Estimated from LLM input size	Every turn	Kill + respawn
denominator drift	2^{48}	B134 vdr denom bits on working values	Every 10 turns	Reproject or kill
error rate	5%	B304 counter get on error/total ratio	Every turn	Kill + respawn
stall count	5	B304 counter get on steps since evidence	Every turn	Backtrack first, then kill

P.2: Clone Lifecycle Token Costs

Phase	Operation	Token Cost	Frequency
Snapshot creation	B368 session snapshot	8 (command token)	Once per stable state
Clone spawn	B371 session clone	8	Per clone lifecycle
Constraint check	B385 constraint check	0 (automated)	Every turn
Clone kill	B372 session kill	8	Per clone lifecycle
Respawn from snapshot	B369 session restore + B371 session clone	16	Per clone lifecycle
Lifecycle overhead	—	40 tokens	Per clone lifetime

40 tokens of overhead per clone lifecycle. If a clone lives for 50 turns at 210 tokens per turn, the lifecycle overhead is 0.38% of the clone’s total token budget. Negligible.

P.3: Knowledge Accumulation Across Clones

Clone	Turns	Persistent Facts Added	Persistent Facts Available	Live State at Kill	Live State Lost
Clone 1	50	35	35	21 KB	Discarded
Clone 2	50	28	63	22 KB	Discarded
Clone 3	50	22	85	20 KB	Discarded
Clone 4	50	18	103	21 KB	Discarded
Total	200	103	103	—	~84 KB total

After 200 turns across 4 clone lifetimes: 103 persistent facts accumulated, each with full provenance. A conventional LLM at turn 200 would have processed over 600,000 tokens of context with severe degradation. The VDR system produced 200 turns at consistent quality with 42,000 total LLM tokens and a growing, queryable knowledge base.

Appendix Q: Operational Environment Cost Comparison

Q.1: Environment Overhead per Use Case

Environment	Startup Cost	Per-Operation Overhead	Isolation Level	Use Cases
Local	0	0	None	Trusted development, KB-only operations
Docker	~2 seconds	~50ms per command	Strong	Default sandbox: scripts, parsing, analysis
SSH remote	~1 second (connection)	~100ms per command (network)	Medium	GPU servers, remote builds
VM	~30 seconds	~200ms per command	Strongest	Untrusted code execution

Q.2: Script Execution Token Cost

Script Type	LLM Tokens to Write	Execution Time	Amortized ReuseToken Cost	Conventional Equivalent
docx parser (5 lines)	30	~500ms	150+ 0.2 per use (per es-say)	N/A (cannot execute)
Migration analyzer (20 lines)	50	~2s for 200 files	1 50 (one-time) (batch)	N/A (cannot execute)
Import graph builder (15 lines)	50	~1s for 200 files	1 50 (one-time) (batch)	N/A (cannot execute)
Metric aggregation (10 lines)	35	~200ms	10+ 3.5 per use	500+ tokens of manual computation
Data validator (8 lines)	25	~100ms	50+ 0.5 per use	200+ tokens of manual checking

Scripts are written once by the LLM (judgment work) and executed by the environment (zero LLM tokens per execution). The conventional LLM cannot execute scripts at all — every computation the script performs would need to be done through token generation.

Appendix R: Knowability Impact on Token Cost

R.1: Source Confidence vs Token Cost to Verify

Source Type	Confidence	Tokens to Verify (Conventional)	Tokens to Verify (VDR)	Verification Mechanism (VDR)
Exact VDR computation	1/1	N/A (cannot verify)	0	Exact by construction
Prolog derivation	1/1	N/A (no Prolog)	0	Derivation chain in KB
Database query	98/100	200+ (re-query, compare)	8 (B378 kb query)	Integer-addressed lookup
Prometheus metric	95/100	500+ (paste, re-read)	8 (B424 network fetch)	Direct fetch with provenance
Python script	95/100	N/A (cannot run)	8 (B410 execute)	Sandbox execution
REST API	85/100	300+ (paste response)	8 (B424 network fetch)	Fetch + parse + convert
User-stated fact	70/100	100+ (recap and confirm)	8 (B376 kb assert)	Stored with confidence tag
Web search	50/100	500+ (search, read, summarize)	40+ (search + parse)	Multiple sources, conflict detection
LLM assessment	30/100	Cannot self-verify	0 (tagged automatically)	Fixed at 30/100 by declaration

The highest-confidence sources (exact computation, Prolog derivation) have zero token cost to verify in VDR because verification is structural. The conventional LLM cannot verify any of its own computations at any token cost.

R.2: Confidence Propagation Token Cost

Propagation Step	Formula	VDR Token Cost	Conventional Token Cost
Independent agreement	$1 - \prod(1 - C_i)$	16 (B003, B002 per source)	100+ tokens of hedging prose
Source conflict	$\max(C_i) - \text{penalty}$	16 (B022, B002)	200+ tokens of “on one hand... on the other hand”
Deductive chain	$\min(C_1 \dots C_n)$	8 per step (B021)	50+ tokens per step of reasoning prose
Hearsay degradation	$\prod(\text{link confidences})$	8 per link (B003)	Cannot compute; generates vague hedging
Inductive scoring	$\text{coverage} \times \text{mean}(C_i)$	24 (B049, B003)	Cannot compute; “some evidence suggests”

Every confidence computation that the conventional LLM expresses as hedging prose — costing 50 to 200 tokens of imprecise language — is a single primitive call in VDR producing an exact fraction.

Appendix S: Token Flow Diagrams

S.1: Conventional Prompt Flow Token Accumulation

Stage	New Tokens Generated	Cumulative Tokens in Context	Purpose
User message arrives	0	Prior history + new message	—
Attention processes full context	0 generated, full context processed	All prior tokens	Re-read everything
Model generates state recap	200	+200	“As we discussed...”
Model parses pasted data	0 explicit, attention over raw data	+0 generated, but data in context	Implicit parsing

Stage	New Tokens Generated	Cumulative Tokens in Context	Purpose
Model gener-ates compu-tation	500	+500	Digit prediction for arithmetic
Model gener-ates deduc-tion	400	+400	Reasoning chain in prose
Model gener-ates results	300	+300	Findings expressed in text
Model gener-ates format-ting	600	+600	Tables, JSON, structure
Model gener-ates hedging	200	+200	“Approximately,” “it seems”
Model gener-ates prose	300	+300	Actual content for user
Turn total	2,500 generated	+2,500 to history	—

Next turn starts with 2,500 more tokens in context. Attention cost increases. Cycle repeats.

S.2: VDR-LLM-Prolog Prompt Flow Token Accumulation

Stage	LLM Tokens	KB Operations	Context Impact	Purpose
User message arrives	0	Cleanup primitives fire	Message cleaned and classified	Automated
Scope resolution	0	Path registry lookup	References resolved to IDs	Automated

Stage	LLM Tokens	KB Operations	Context Impact	Purpose
Data fetching	0	KB query + primitive reads	Data in typed containers	Automated
Grammar parsing	0	Grammar recognizes structure	Typed fields extracted	Automated
Context assembly	0	Structured summary built	Compact typed input ready	Automated
LLM assesses	30	Reads structured state	Judgment on clean input	LLM work
LLM formalizes step	20	—	Command token composed	LLM work
Primitive executes	0	Result stored in KB	Finding with provenance	Automated
LLM writes prose	80	Grammar provides structure	Content slots filled	LLM work
Confidence computed	0	Propagation primitives	Exact fraction stored	Automated
Turn total	130 LLM tokens	~15 KB operations	+0 to context history	—

Next turn starts with identical context size. State changes are in KBs, not context. No accumulation in the token stream.

Appendix T: Error Class Elimination Matrix

T.1: Error Types by System Component

Error Class	Conventional Source	Conventional Rate	VDR Elimination Mechanism	VDR Rate	Builtin Category
Integer arithmetic	Digit prediction	~2% per operation	B139-B159 integer fast path	0	Exact by construction
Rational arithmetic	Digit prediction	~5% per multi-step chain	B001-B008 VDR closed	0	Exact by construction
Comparison	Token generation reasoning	~3% per comparison	B017-B026 comparison	0	Cross-multiply integers

Error Class	Conventional Source	Conventional Rate	VDR Elimination Mechanism	VDR Rate	Builtin Category
Sorting	Generated comparison prose	~8% for >10 items	B202-B204 list sort	0	Deterministic algorithm
Filtering	Attention over data	~5% per filter pass	B198 list filter	0	Predicate evaluation
Aggregation	Digit accumulation	~10% for >20 values	B047-B054 aggregates	0	Exact sum/product
Statistical measures	Multi-step digit prediction	~15% for variance+	B101-B105 statistics	0	Exact arithmetic chain
Deductive inference	Reasoning chain prose	~10% per 5-step chain	Prolog evaluation	0	Structural unification
Fact retrieval	Attention over history	~5% per turn after turn 10	B378 kb query	0	Integer-addressed lookup
State tracking	Context window degradation	~3% per turn cumulative	KB persistence	0	Persistent typed fields
JSON formatting	Structural token prediction	~3% per response	Grammar template	0	Template logic
Table formatting	Structural token prediction	~7% per table	Grammar template	0	Template logic
Cross-reference	Attention across distant context	~15% for >20 page docs	B366 connection list	0	Typed connections
Confidence assessment	Hedging language generation	100% imprecise	Propagation primitives	0 (exact fraction)	Declared rules

T.2: Cumulative Error Probability per Use Case

Use Case	Operations	Conventional Error Probability	VDR Error Probability	Error Surface
SRE (20 comparisons, 5 aggregations, 10 retrievals)	35	$1 - (0.97^{20} \times 0.90^5 \times 0.95^{10}) \approx 73\%$	0% (all primitive)	LLM judgment only
Legal (50 retrievals, 10 arithmetic, 20 cross-refs)	80	$1 - (0.95^{50} \times 0.95^{10} \times 0.85^{20}) \approx 99\%+$	0% (all primitive)	LLM judgment only
Financial (500 multiplications, 500 comparisons, 25 aggregations)	1025	Effectively 100%	0% (all primitive)	LLM judgment only
Grading (150 retrievals, 150 sums, statistics)	300+	Effectively 100%	0% (all primitive)	LLM scoring judgment

For any use case with more than roughly 30 computational or retrieval operations, a conventional LLM is virtually certain to have at least one error. VDR has zero computational errors by construction. The remaining error surface is exclusively LLM judgment — the task where error rates are lowest.

Appendix U: KB Tree Depth and Query Cost

U.1: Typical Tree Structures by Use Case

Use Case	Tree Depth	Total KBs	Scope Chain Length	Query Resolution Steps	Notes
SRE incident	4	5	4	max 4 integer lookups	root → sessions → incident → findings
Legal contract	5	8	5	max 5	root → sessions → review → clauses → terms
Medical synthesis	4	42	4	max 4	root → sessions → synthesis → papers (40 siblings)
Codebase migration	4	205	4	max 4	root → sessions → migration → files (200 siblings)

Use Case	Tree Depth	Total KBs	Scope Chain Length	Query Resolution Steps	Notes
Financial portfolio	4	10	4	max 4	root → ses- sions → port- folio → analy- sis
Support KB	4	2003	4	max 4	root → knowl- edge → sup- port → ar- ticles (2000 sib- lings)
Academic grading	4	155	4	max 4	root → ses- sions → grad- ing → stu- dents (150 sib- lings)
Organization (full)	6	500+	6	max 6	root → org → dept → team → user → ses- sion

U.2: Query Cost by Operation

Operation	Mechanism	Cost (integer ops)	Scaling	Equivalent LLM Tokens
Path resolve	Hash lookup + array index	2-6 (per segment)	$O(\text{depth})$	0 (automated)
Fact query in single KB	Linear scan or hash on predicate	1-50 (depends on fact count)	$O(\text{facts per KB})$	200+ (attention search)
Scoped query (walk ancestors)	Repeat per scope level	$4-24 (\text{depth} \times \text{per-KB cost})$	$O(\text{depth} \times \text{facts})$	500+ (full context search)
Cross-scope query (B380)	Scan all KBs	Varies	$O(\text{total KBs} \times \text{facts})$	Cannot perform
Connection traversal	Array index per hop	2 per hop	$O(\text{hops})$	Cannot perform
Mount resolution	Follow chain, cycle check	3-10	$O(\text{chain length})$	Cannot perform

Even the most expensive KB query — a cross-scope scan across 2000 KBs — involves integer operations measured in thousands. One LLM token costs millions of float ops. The entire query infrastructure is computationally invisible relative to a single generated token.

Appendix V: Format-Specific Token Savings Detail

V.1: JSON Output Decomposition

JSON Element	Tokens per Instance	Instances in Typical Object	Total Structural Tokens	Grammar Provides	LLM Generates
Opening brace	1	1	1	Yes	No
Closing brace	1	1	1	Yes	No

JSON Element	Tokens per Instance	Instances in Typical Object	Total Structural Tokens	Grammar Provides	LLM Generates
Key quotation marks	2	20	40	Yes	No
Key names	1-3	20	40	Yes (from schema)	No
Colons	1	20	20	Yes	No
Commas	1	19	19	Yes	No
Value quotation marks	2	12 (string values)	24	Yes	No
Whitespace/newlines		20	20	Yes	No
Structural subtotal	—	—	165	165	0
String values	2-5	12	42	No	Yes
Numeric values	1-3	8	16	No (but from KB)	Partially
Content subtotal	—	—	58	0-16	42-58
Total	—	—	223	165-181	42-58

Grammar provides 74-81% of tokens in a typical JSON response.

V.2: Markdown Table Decomposition (20 rows × 5 columns)

Table Element	Tokens	Count	Total	Grammar	LLM
Header row pipes	1 each	6	6	Yes	No
Header cell content	1-2 each	5	8	Yes (from schema)	No
Separator row	1 per cell + pipes	11	11	Yes	No
Data row pipes	1 each	6 per row × 20	120	Yes	No
Row newlines	1 each	22	22	Yes	No
Structural subtotal	—	—	167	167	0
Cell content	1-3 each	100	200	From KB if data	Prose only
Total	—	—	367	167-367	0-200

For data tables (all cells from KB): grammar provides 100%. LLM generates 0 tokens.
 For mixed tables (some prose cells): grammar provides 45-100%. Either way, structural correctness is guaranteed.

Appendix W: Cross-Paper Validation Coverage

W.1: Claims Made in VDR-15 and Their Validation Source

Claim	Section	Validation Source	Test Count	Status
VDR arithmetic is exact	4, 8	VDR-1 through VDR-3	507	Zero errors
Softmax sums to exactly one	9	VDR-4 Batch 3	5 configurations tested	Exact 1 in all cases
Exact gradients (autodiff)	8	VDR-4 C08	12 tests	Exact derivatives
KB scoping prevents ambiguity	2, 8	VDR-5	Design property	Structural
Data primitives bounded	Appendix I	VDR-8	Per-type tests	Capacity enforced
Snapshots atomic	Appendix P	VDR-8	Snapshot/restore tests	Bit-identical restore
Grammar bidirectional	10	VDR-12	178/179	Roundtrip verified
Compression ~83%	10	VDR-12	150,000 words tested	Verified per paper
Conservation laws exact	8	VDR-13	10 laws, 14 domains	Structural equality
Float fails where VDR succeeds	4, 8	VDR-13	12 failure points	Documented

Claim	Section	Validation Source	Test Count	Status
Orchestrated 7.1 inference loop		VDR-9	Design property	Pattern over existing spec
Confidence propagation	Appendix R	VDR-9	Formula specification	Exact arithmetic
Command tokens ~8 tokens	3, Appendix J	VDR-6, VDR-8	Design specification	Structural
448 builtins with IOSE	Appendix H	VDR-6, VDR-10, VDR-13	533 IOSE declarations	Complete
Grant default denial	Appendix O	VDR-6	Design property	Structural
Q335 ~100 digit precision	4	VDR-3	22 constants verified	Numerator bit widths 330-334

W.2: New Analysis in VDR-15 Not Present in Prior Papers

Analysis	Section	Nature	Basis
Token category taxonomy (judgment/command/infrastructure)	3	Classification framework	Novel to VDR-15
Per-use-case token count comparison	7	Quantitative estimation	Based on VDR-14 architecture
Crossover calculation (Q335 slowdown budget)	4	Mathematical analysis	VDR-3 Q335 costs + LLM token costs
Conversation length scaling (quadratic vs linear)	5	Scaling analysis	VDR-8 session model + attention mechanics
Capability boundary enumeration	6	Architectural analysis	VDR-6 primitive model + context window limits

Analysis	Section	Nature	Basis
Context window utilization breakdown	Appendix K	Quantitative model	Novel to VDR-15
Command token entropy analysis	Appendix J	Information-theoretic analysis	VDR-8 command token spec
Error class elimination matrix	Appendix T	Reliability analysis	VDR-1 through VDR-13 test results
Grammar amortization economics	Appendix L	Cost analysis	VDR-12 grammar specification
Prolog rule composition economics	Appendix M	Cost analysis	VDR-5 Prolog specification
Disposable clone accumulation model	Appendix P	Lifecycle analysis	VDR-8 clone specification
Data primitive memory footprint	Appendix I	Memory analysis	VDR-8 primitive specification
Operational environment cost model	Appendix Q	Performance analysis	VDR-6 environment specification

VDR-15 introduces no new primitives, builtins, struct fields, or modules. Every analysis is a pattern-of-use examination over the existing specification, demonstrating emergent properties of the composed system.

Appendix X: Workday Economics Extended

X.1: Legal Team Workday

Task	Count	Conventional Tokens	VDR Tokens	Conventional Quality	VDR Quality
Contract3 re-view		90,000	3,390	Cross-refs lost	Full cross-ref

Task	Count	Conventional Tokens	VDR Tokens	Conventional Quality	VDR Quality
Clause comparison (2 contracts)	2	20,000	800	Manual side-by-side	Exact diff via primitives
Regulatory compliance check	5	25,000	2,500	Training data recall	KB query against regulation DB
Memo drafting	4	12,000	4,000	Generic templates	Grammar + KB-sourced facts
Client Q&A	10	5,000	1,000	Generic answers	KB-backed with provenance
Day total	24	152,000	11,690	—	—

Ratio: 13:1. Plus qualitative improvements in every task.

X.2: Research Team Workday

Task	Count	Conventional Tokens	VDR Tokens	Notes
Literature synthesis	1 (40 papers)	80,000	4,740	Full statistical aggregation in VDR
Data analysis	3 datasets	45,000	1,800	Exact statistics in VDR
Hypothesis testing	5 hypotheses	15,000	300	Prolog rules in VDR
Report drafting	2 reports	10,000	3,000	Grammar + KB data in VDR

Task	Count	Conventional Tokens	VDR Tokens	Notes
Peer discussion prep	3	6,000	900	KB-sourced talking points
Day total	—	156,000	10,740	—

Ratio: 14.5:1.

X.3: Development Team Workday

Task	Count	Conventional Tokens	VDR Tokens	Notes
Code review	10 PRs	30,000	5,000	File read + analysis script in VDR
Bug investigation	3 bugs	45,000	3,000	Abductive template + test execution
Feature implementation	2 features	20,000	8,000	Code gen is LLM judgment work
Migration work	50 files	25,000	1,675	Pattern from Section 7.4 scaled
Documentation	5 docs	10,000	2,000	Grammar templates + KB content
Day total	—	130,000	19,675	—

Ratio: 6.6:1. Lower ratio because code generation is irreducibly LLM judgment work. But bug investigation and migration — the most error-prone tasks — show the highest ratios.

X.4: Cross-Domain Summary

Domain	Daily Conventional Tokens	Daily VDR Tokens	Ratio	Highest-Ratio Task	Lowest-Ratio Task
SRE	640,000	26,000	24.6:1	Postmortem (120:1)	Ad-hoc query (5:1)
Legal	152,000	11,690	13.0:1	Contract review (26.5:1)	Memo drafting (3:1)
Research	156,000	10,740	14.5:1	Synthesis (16.9:1)	Report drafting (3.3:1)
Development	130,000	19,675	6.6:1	Migration (14.9:1)	Feature impl (2.5:1)
Finance	200,000	15,000	13.3:1	Portfolio analysis (25:1)	Commentary (3:1)
Education	250,000	60,000	4.2:1	Statistics (∞ — cannot do)	Essay grading (3.5:1)
Support	50,000	8,000	6.3:1	KB query (3.3:1)	Setup (2500:1 amortized)

Pattern: tasks dominated by data processing, computation, and state management show ratios of 10:1 to 100:1 or higher. Tasks dominated by prose generation (memos, feature implementation, commentary) show ratios of 2:1 to 4:1. The floor is never below 2:1 because grammar always saves structural tokens on output.

[[HOWL-VDR-15-2026](#)] VDR-LLM-Prolog: Prompt Optimization. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-15-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-3-2026](#)] VDR Gym Extension. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-3-2026>

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026>

[[HOWL-LLM-1-2026](#)] Integer LLM with Prolog Knowledge Base. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026>

[[HOWL-VDR-5-2026](#)] Exact Arithmetic Meets Logical Provenance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026>

[[HOWL-VDR-6-2026](#)] Computational Primitives and Operational Environments. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-6-2026>

[[HOWL-VDR-7-2026](#)] Complete Lifecycle Technical Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-7-2026>

[[HOWL-VDR-8-2026](#)] Computational State Primitives, Universal Data Pathing, and Session Management. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026>

[[HOWL-VDR-9-2026](#)] Orchestrated Inference. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-9-2026>

[[HOWL-VDR-10-2026](#)] Operational Foundations and Comprehensive Builtin Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-10-2026>

[[HOWL-VDR-11-2026](#)] Implementation Blueprint. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-11-2026>

[[HOWL-VDR-12-2026](#)] Grammar-Directed Compaction and Generation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-12-2026>

[[HOWL-VDR-13-2026](#)] VDR in Physical Computation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-13-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>